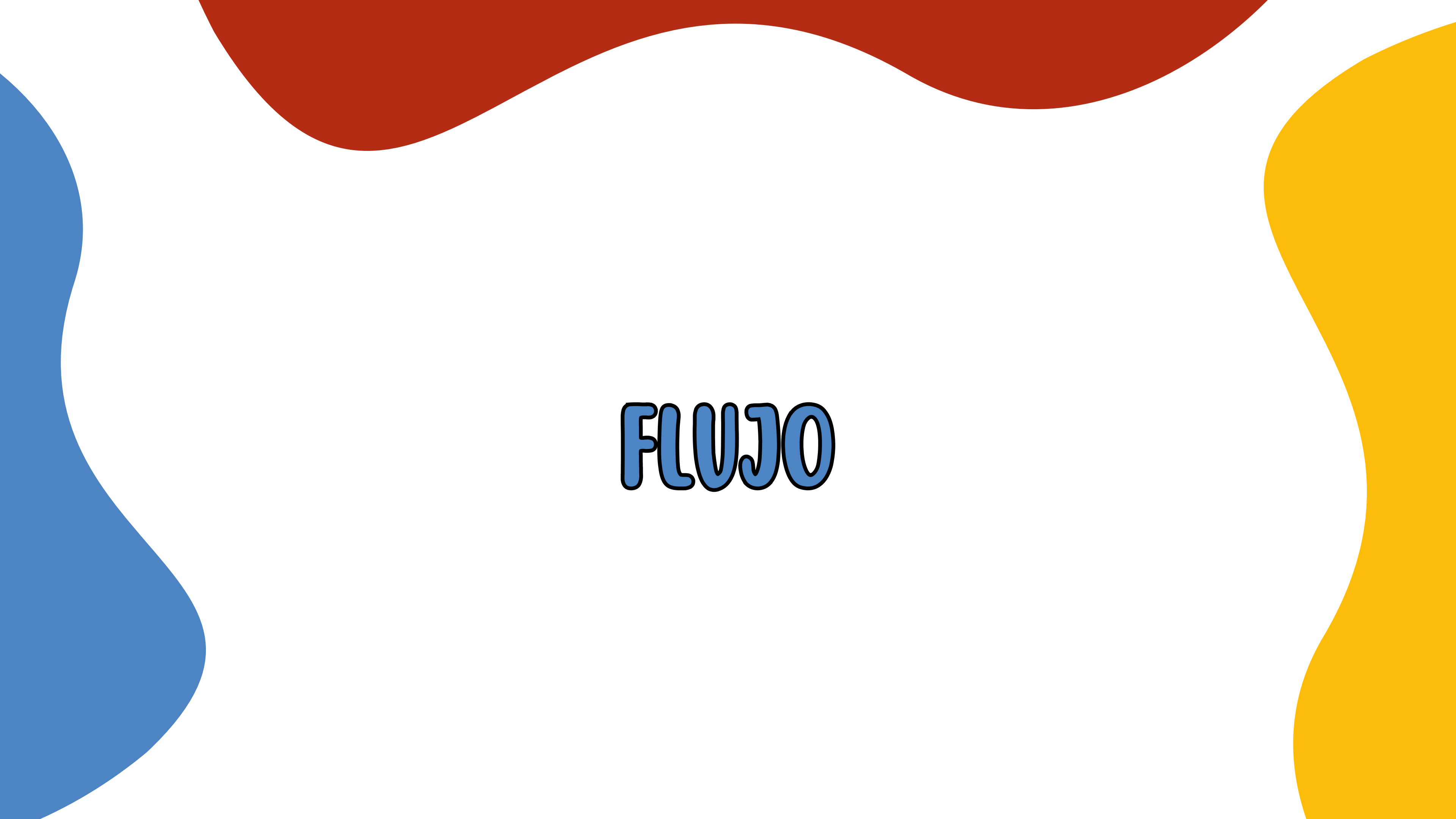
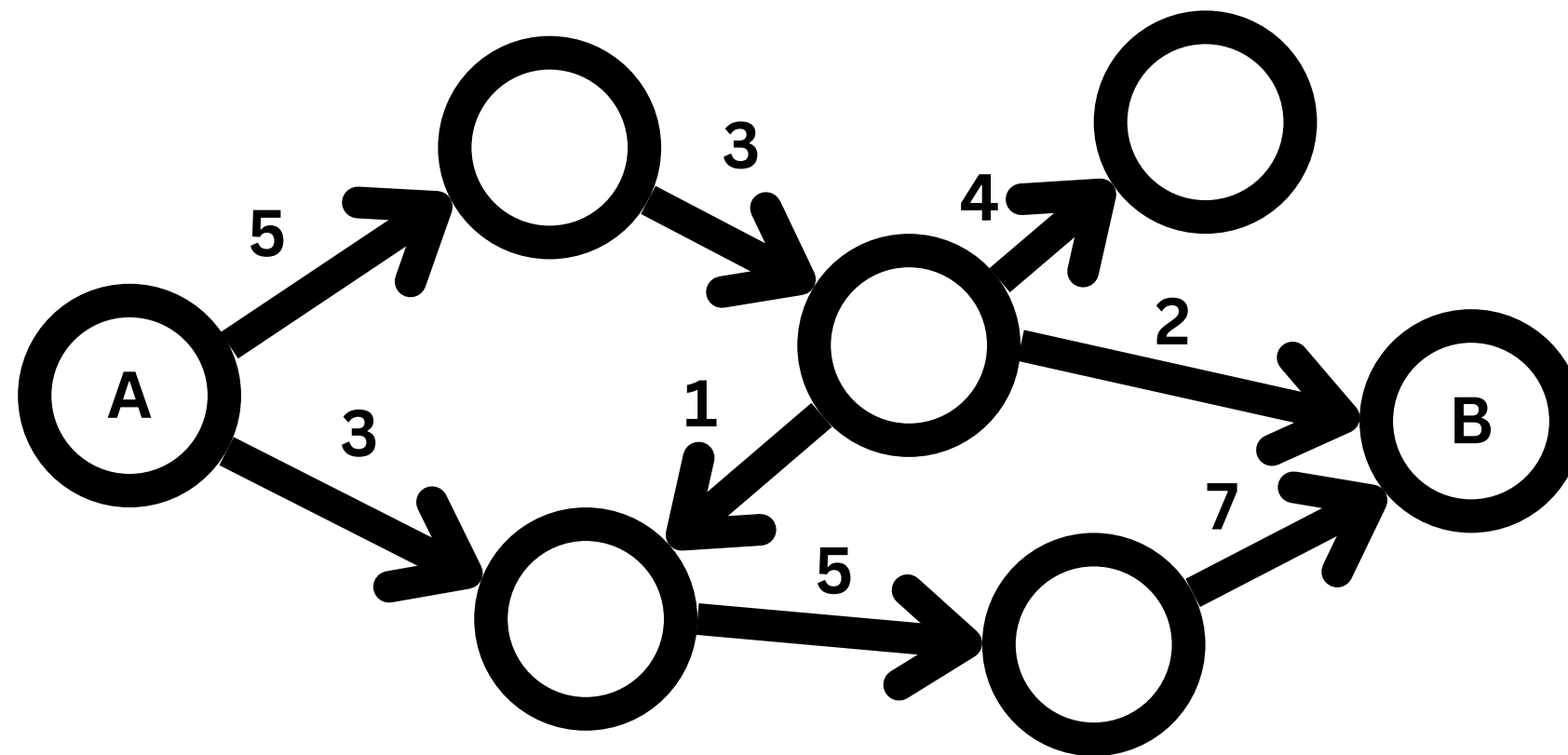




FLUJO

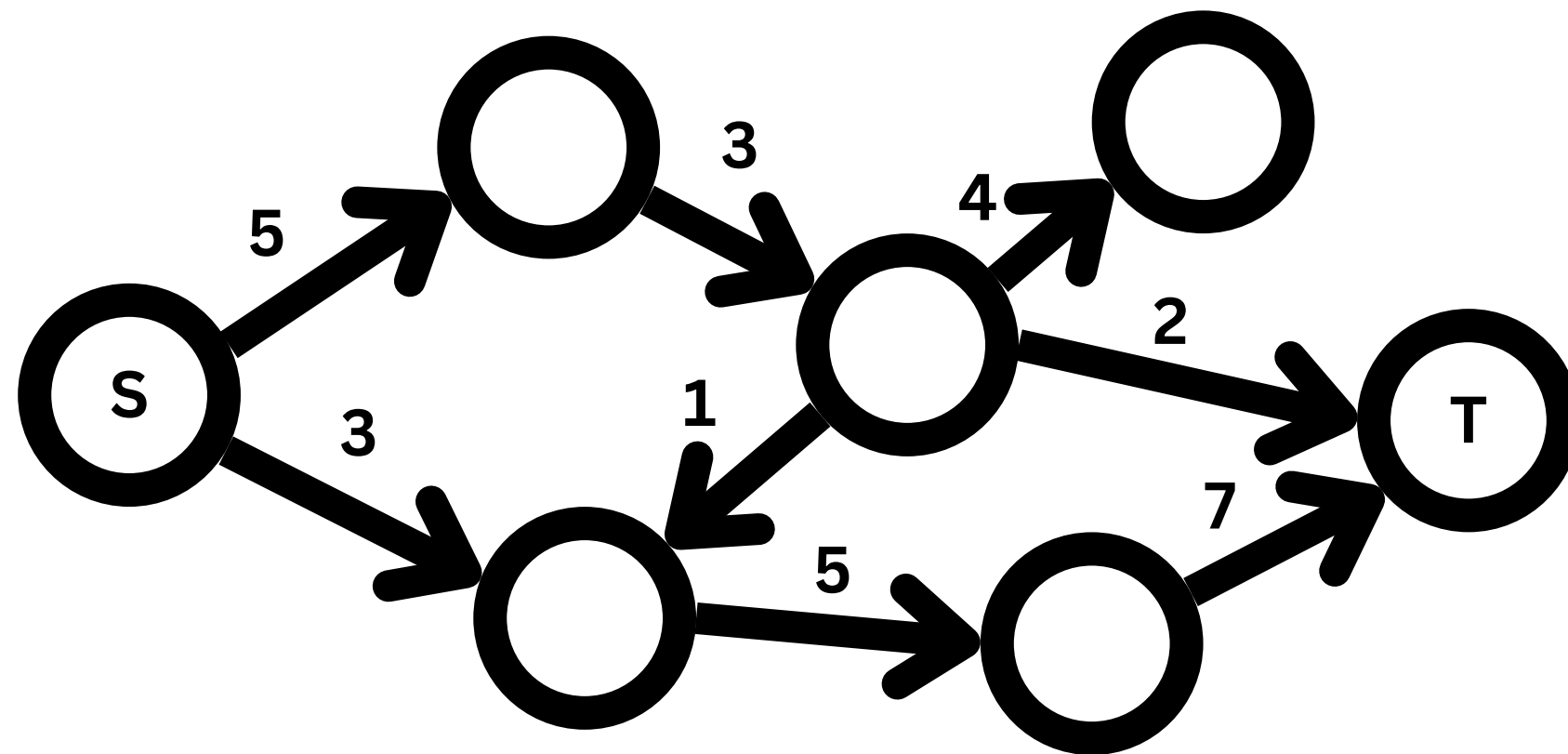
PROBLEMA

- Tenemos un sistema de carreteras.
- Cada carretera tiene un flujo máximo de camiones por hora.
- ¿Cuántos camiones podemos mover de S a T? ¿Por dónde?



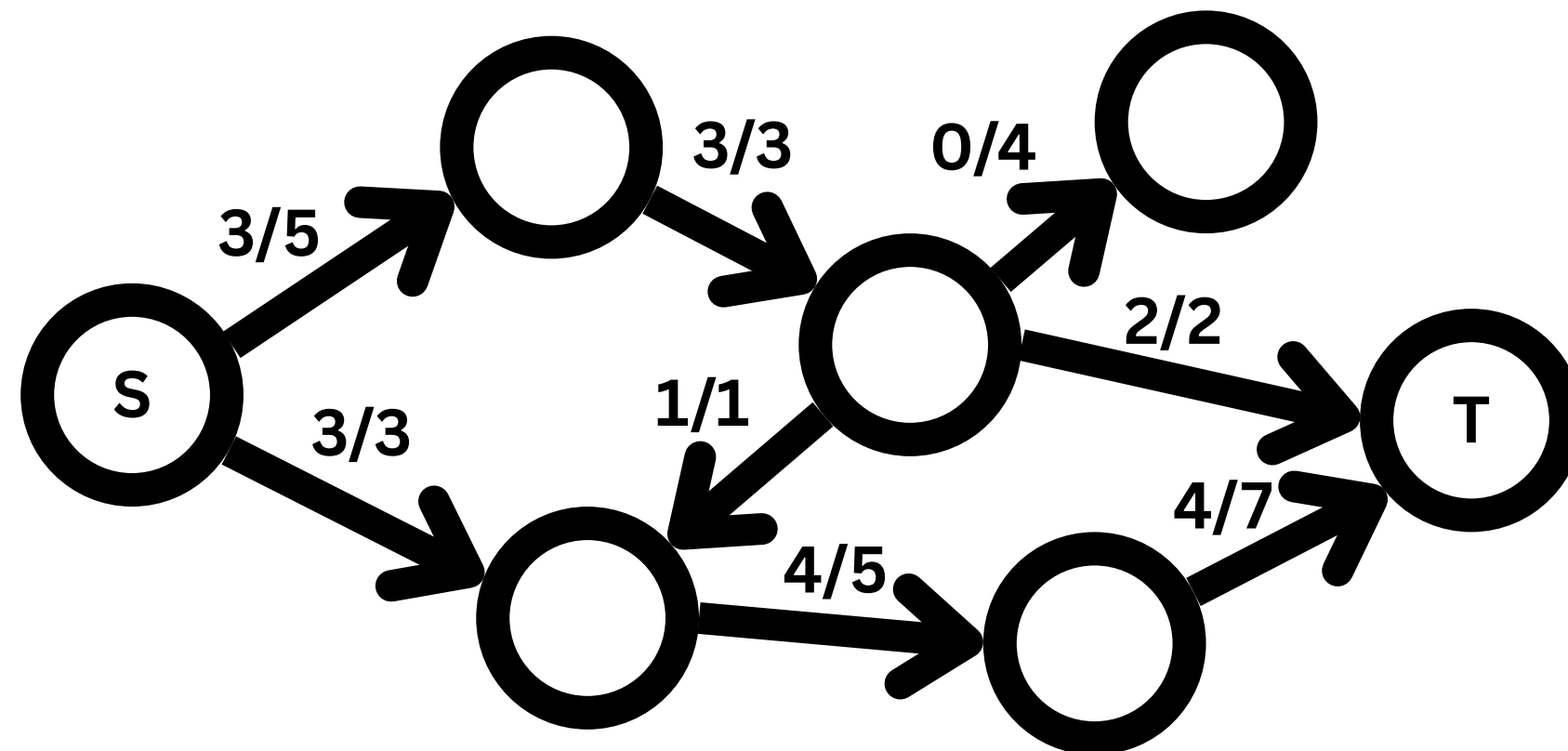
PROBLEMA

- Grafo dirigido
- Aristas con peso: **Capacidad** de cada arista
- Output: Un **"flujo"**



FLUJO

- Un **pre-flujo** es una asignación de un número a cada arista: el **flujo de la arista**
- Un **flujo** es un **pre-flujo** que sea **válido**

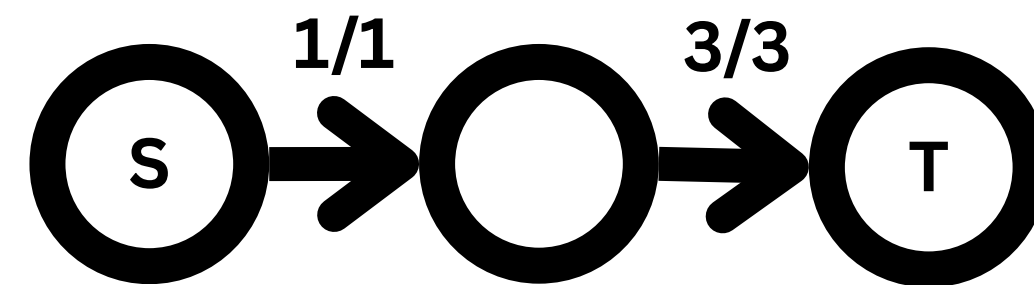


PRE-FLUJO VS FLUJO

- Un **pre-flujo** es una asignación de un número a cada arista: el **flujo de la arista**
- Un **flujo** es un **pre-flujo** que sea **válido**



Pre-flujo inválido
Flujo supera capacidad

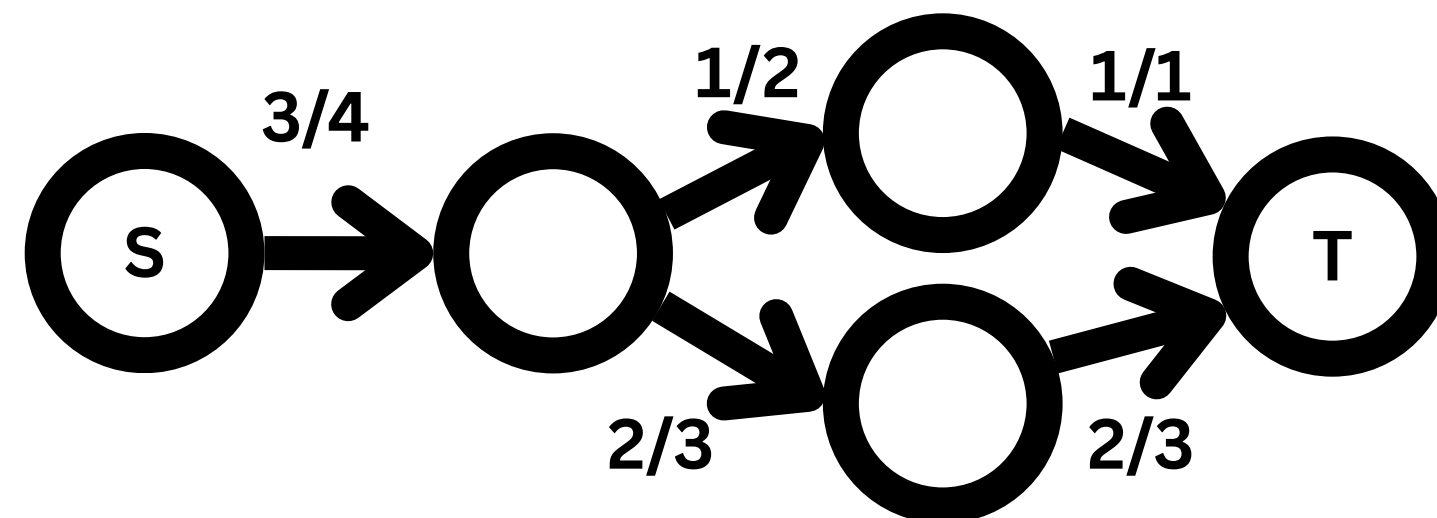


Pre-flujo inválido
No se conserva el flujo

PRE-FLUJO VS FLUJO

Para que un pre-flujo sea válido:

- Cada arista tiene que tener $0 \leq \text{flujo} \leq \text{capacidad}$
- El **flujo neto** (entrante menos saliente) de cada **nodo** debe ser 0
 - Excepción para nodo **S**: flujo neto debe ser no-negativo
 - Excepción para nodo **T**: flujo neto debe ser no-positivo

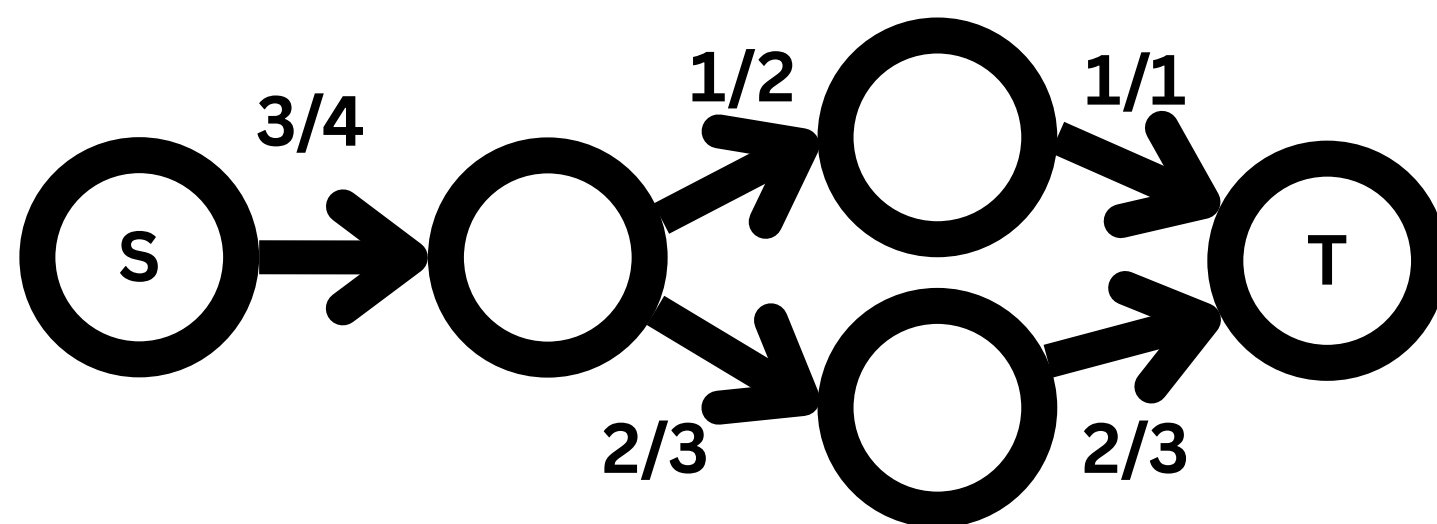


VALOR DE UN FLUJO

Con las restricciones anteriores, el flujo en que por todas las aristas fluye 0 es un flujo válido también. ¿Cómo caracterizamos el flujo “bueno” que buscamos?

Asignamos un **valor** al flujo:

- La cantidad de flujo que sale de S
- La cantidad de flujo que entra a T (equivalente a lo anterior)

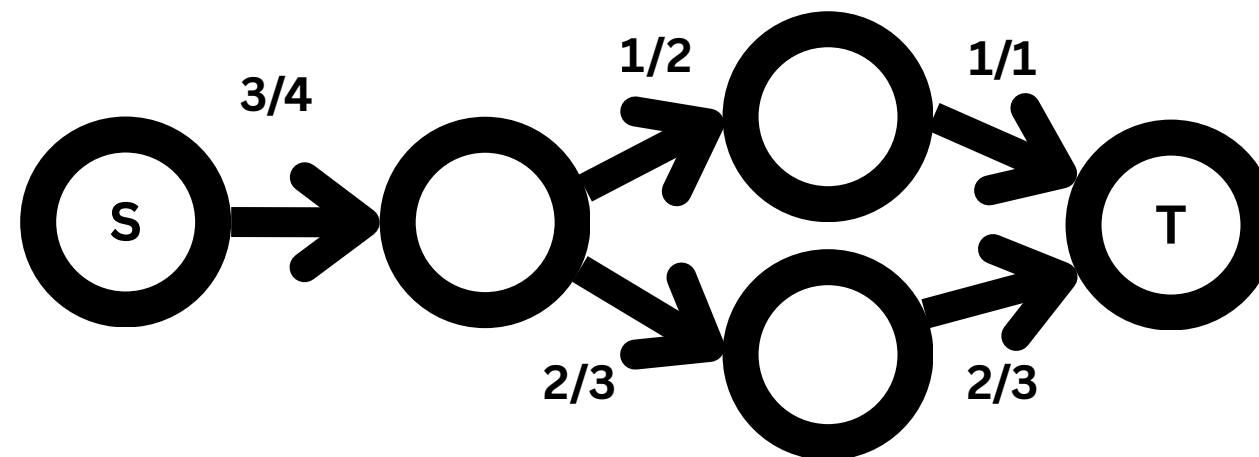


Valor de este flujo: **3**

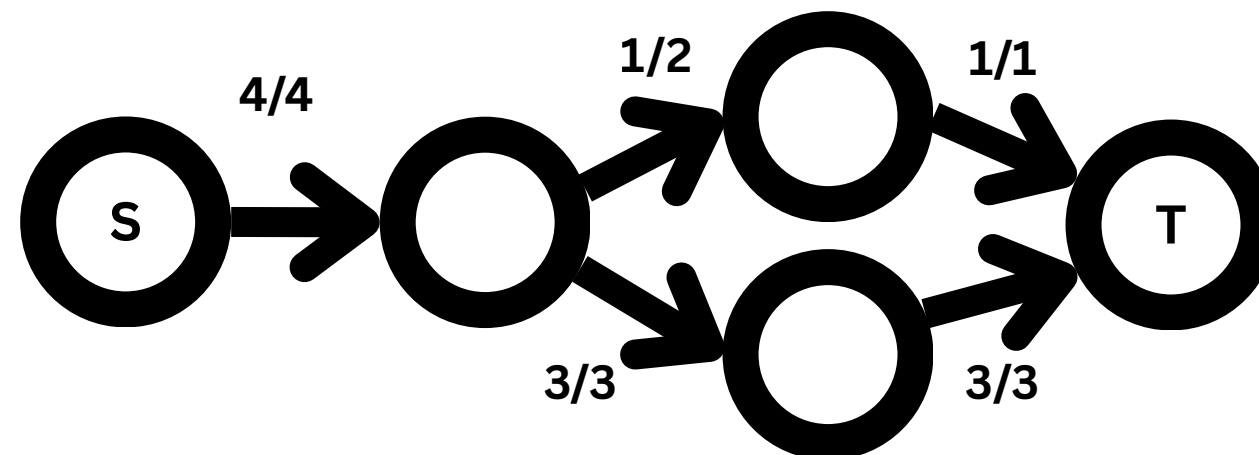
FLUJO MÁXIMO

Finalmente, formalizando el problema original:

- Buscamos, entre todos los flujos, el **flujo** que tenga máximo **valor**.
- El **flujo máximo**.



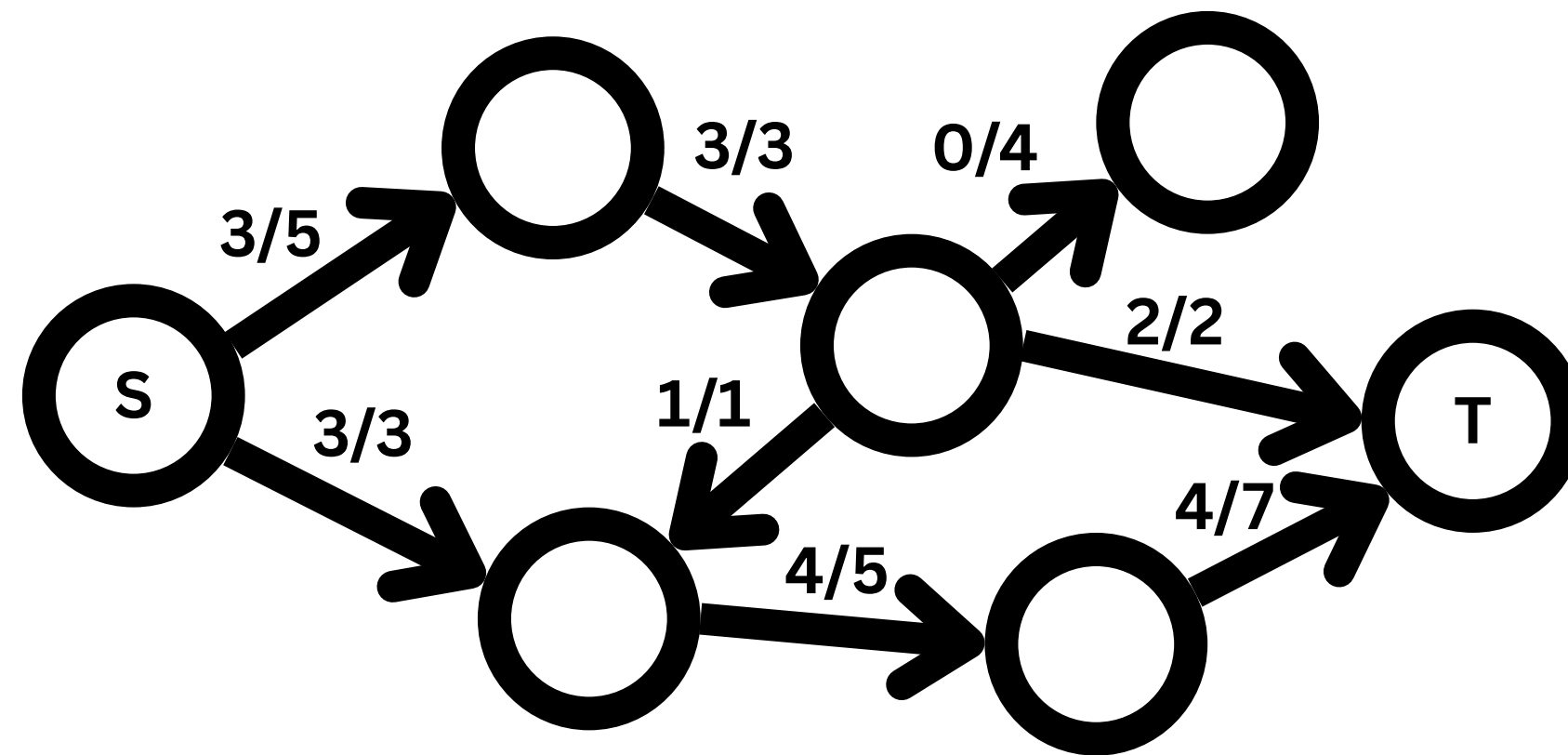
Valor de este flujo: **3**



Valor de este flujo: **4**

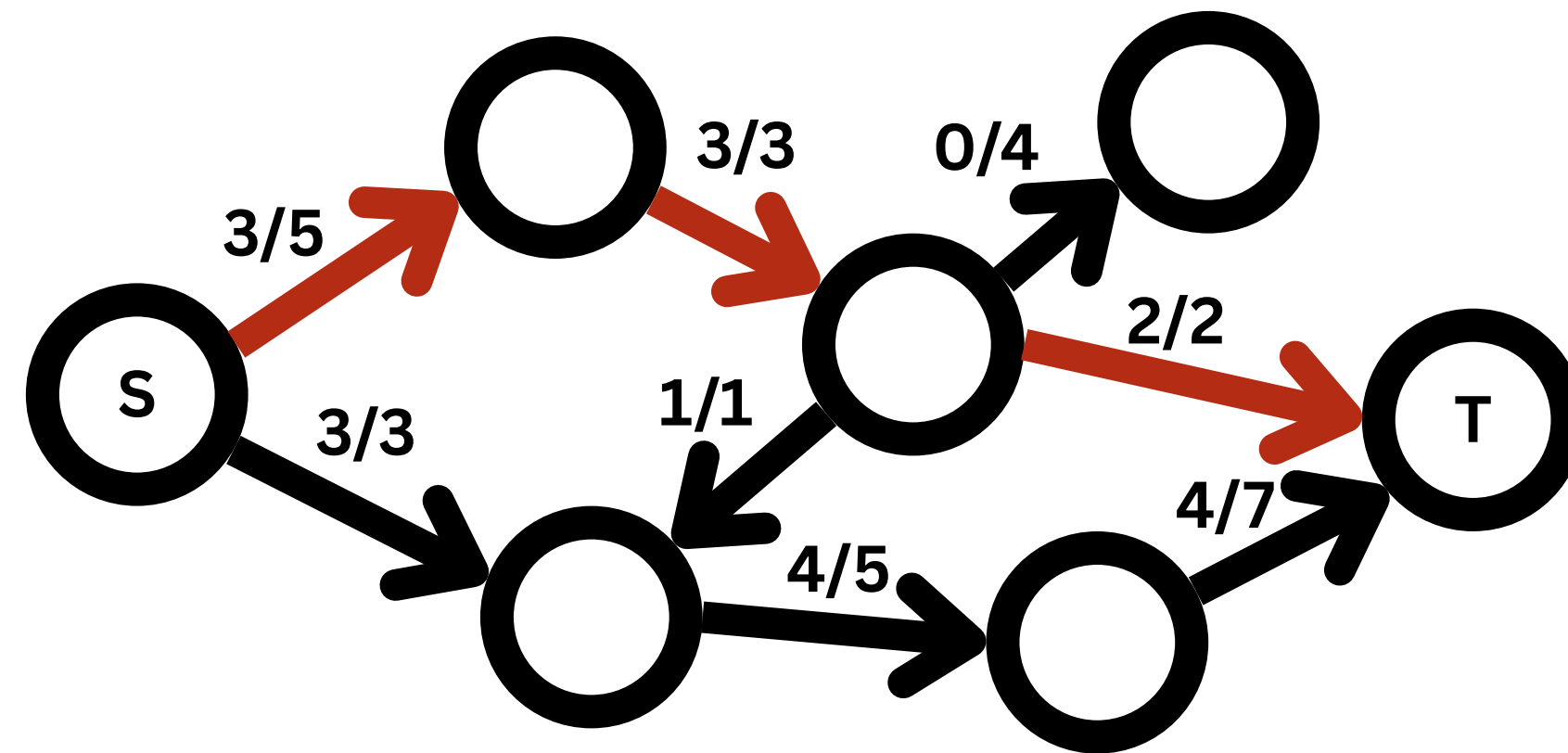
ALGORITMO

¿Cómo encontramos un flujo máximo?



ALGORITMO

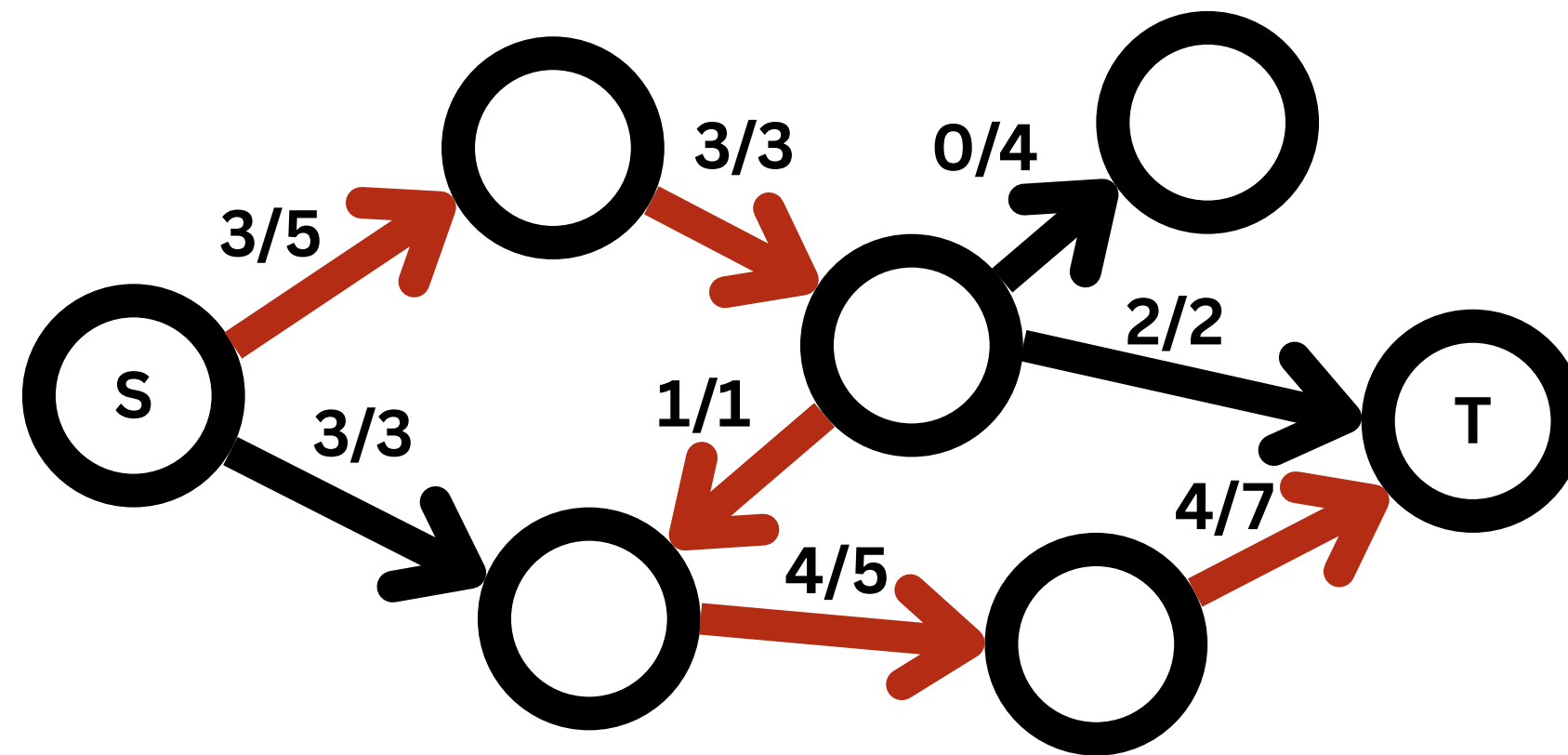
Notemos que el flujo pasa por **caminos** de S a T.



Este camino aporta **2**

ALGORITMO

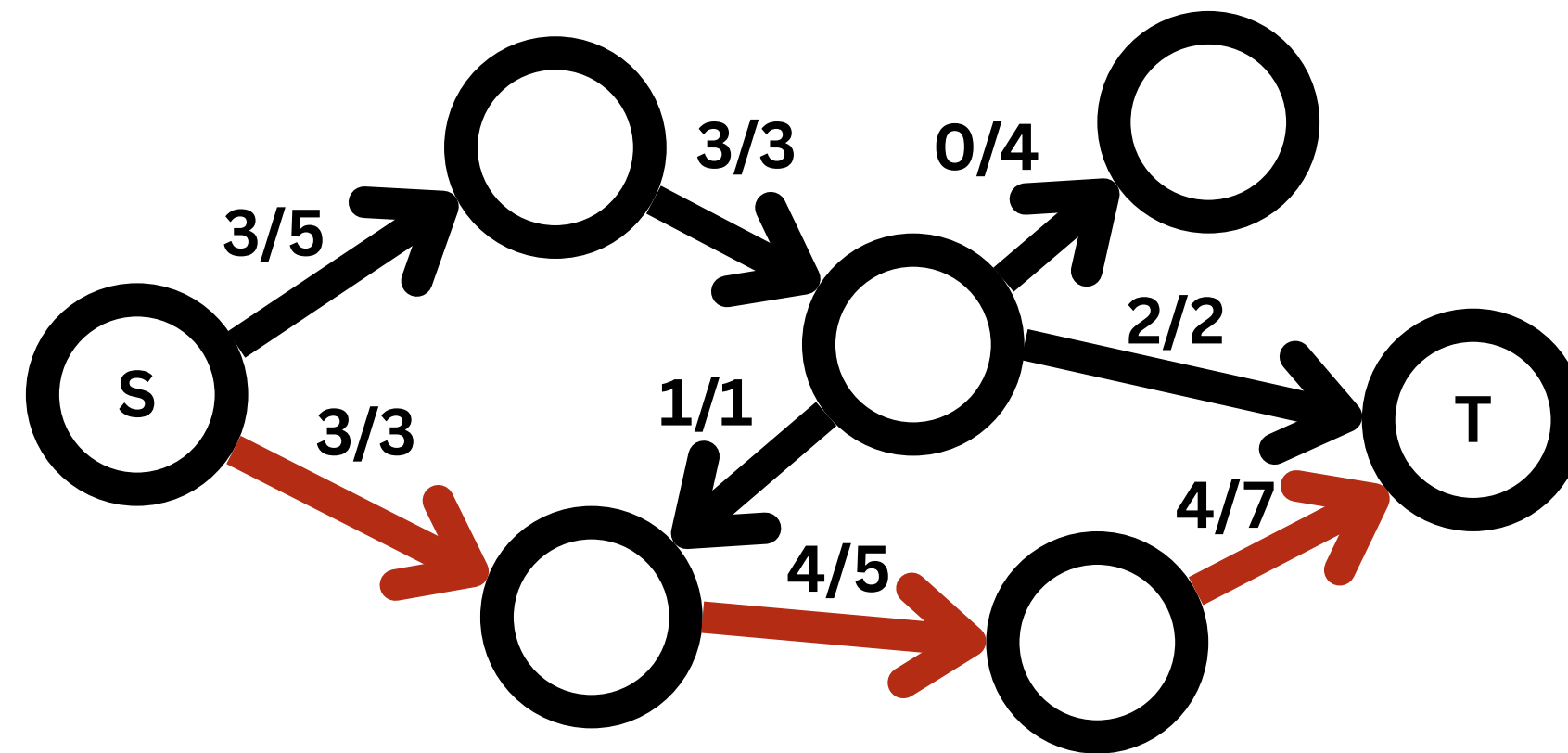
Notemos que el flujo pasa por **caminos** de S a T.



Este camino aporta **1**

ALGORITMO

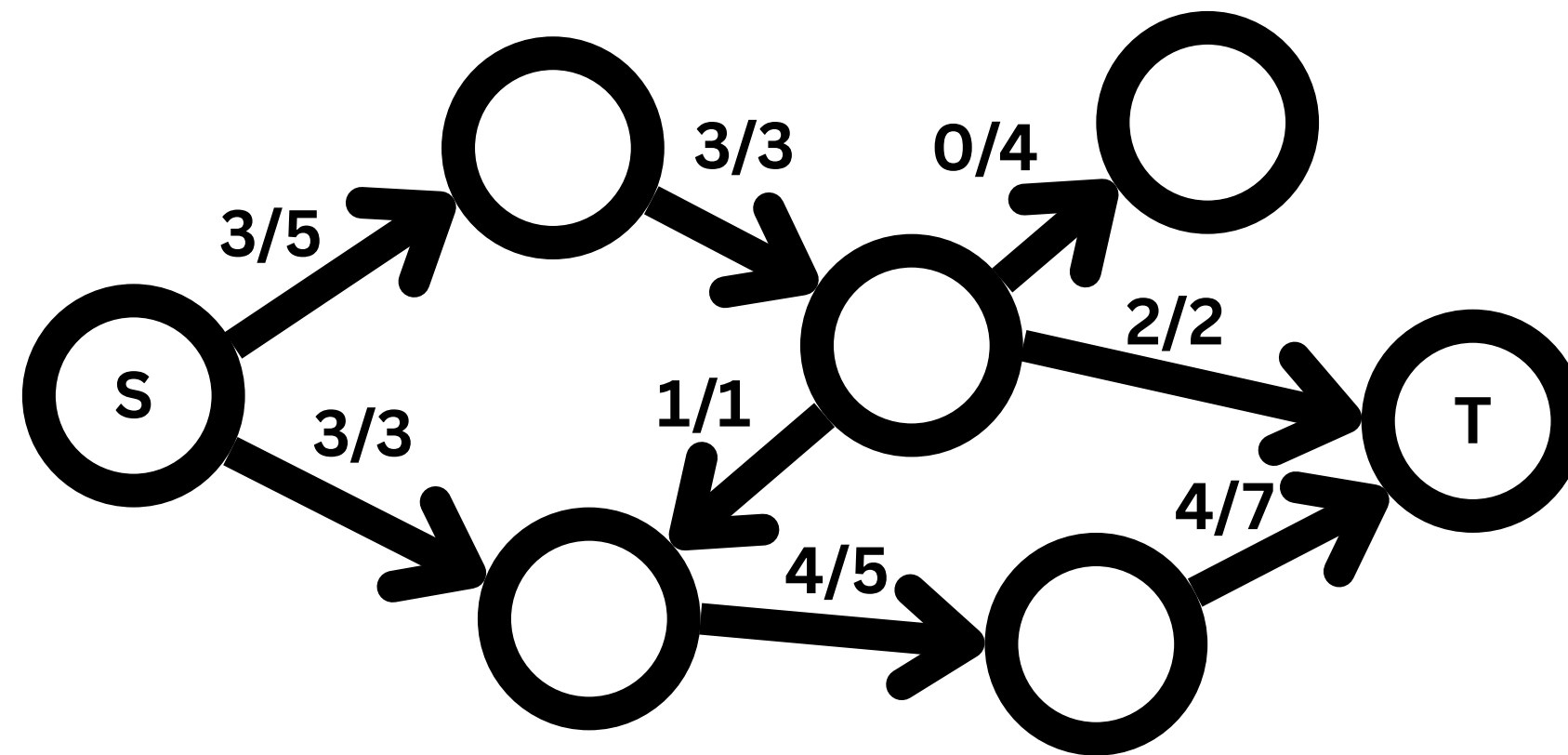
Notemos que el flujo pasa por **caminos** de S a T.



Este camino aporta **3**

ALGORITMO

Tenemos 3 caminos, que juntos componen todo el flujo.
El valor total del flujo es $2 + 1 + 3 = 6$



FORD-FULKERSON

Esta idea lleva a un algoritmo inventado por Ford-Fulkerson en 1956.

1. Construimos un flujo iterativamente. Inicialmente el flujo es 0.
2. Buscar un camino de S a T que tenga capacidad disponible.
3. Encontramos la mínima capacidad disponible en el camino. Agregamos esta capacidad disponible al flujo de las aristas del camino.
4. Volver al paso 2 hasta que no existan caminos con capacidad disponible.

COMPLEJIDAD

Para cada iteración, debemos hacer una búsqueda de un camino entre S y T. La búsqueda la hacemos sobre **la red residual** y tiene complejidad $O(E)$ si la implementamos con un DFS por ejemplo.

Hacemos F iteraciones, donde F es el valor del flujo máximo.

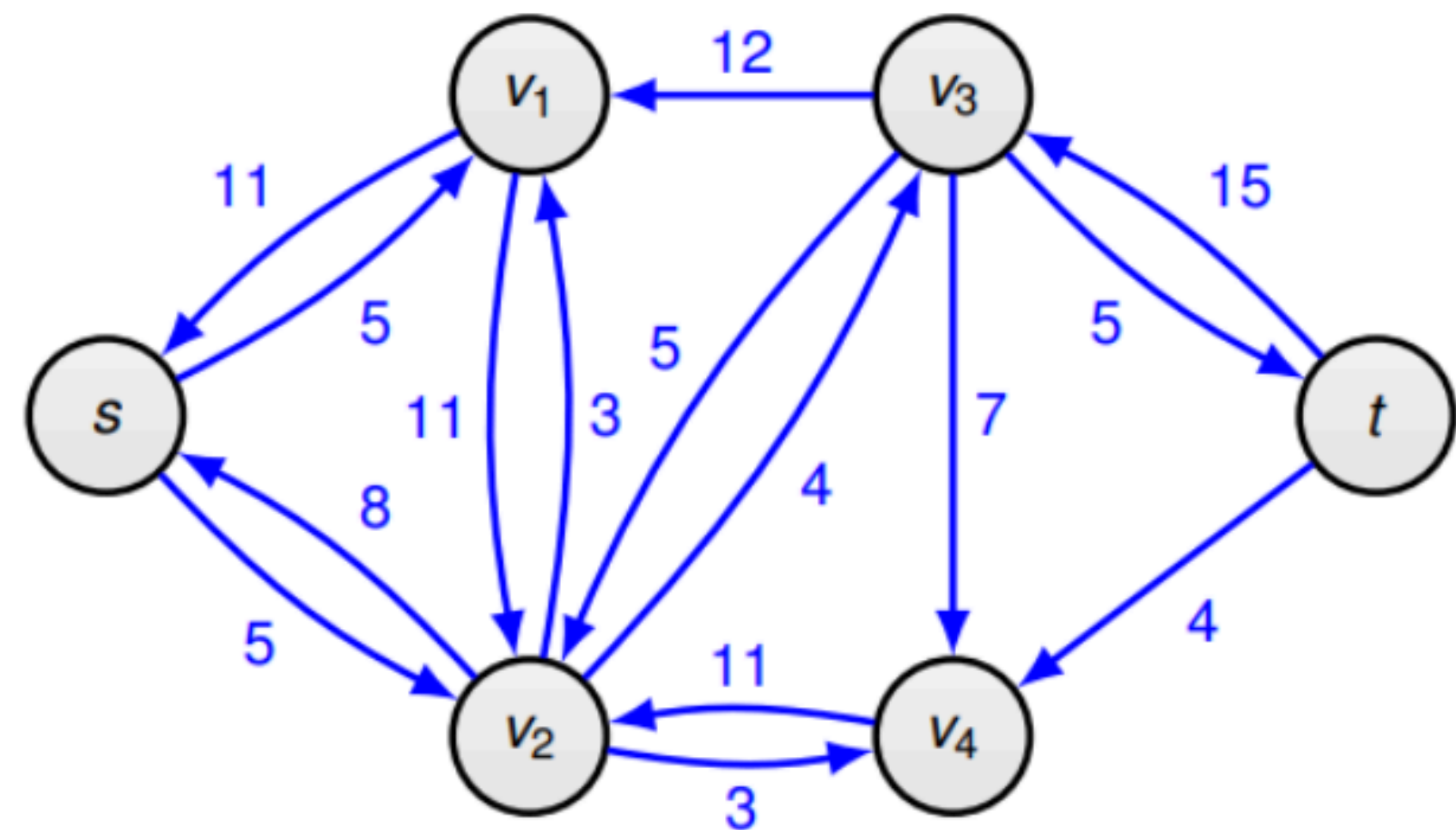
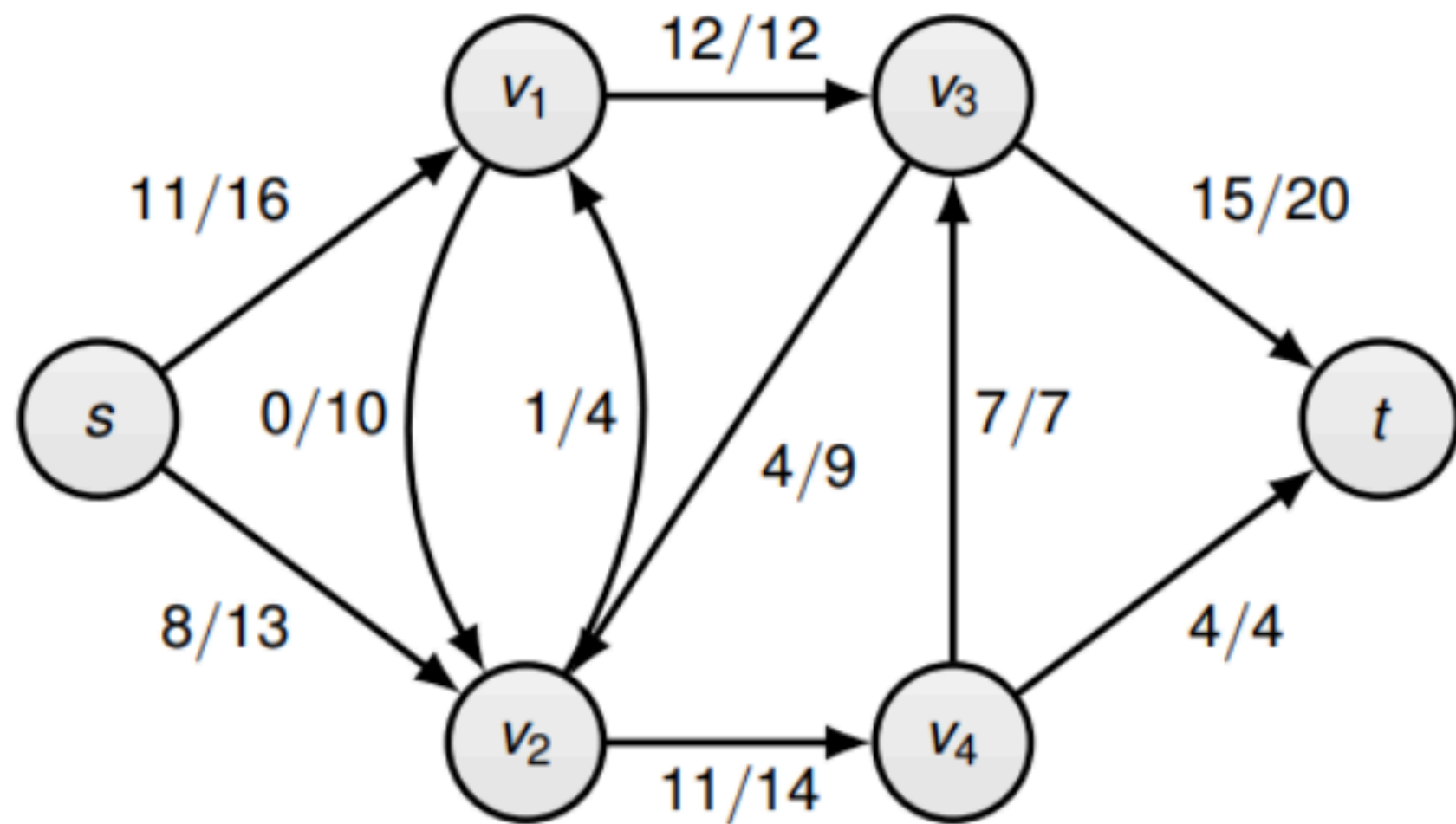
Por ende, la complejidad es $O(E F)$

RED RESIDUAL

La red residual de un flujo es un grafo dirigido tal que:

- Los nodos son los mismos que en la red original.
- Hay una arista de u a v con capacidad la suma de:
 - Lo que me falta para llenar la capacidad de u a v en el grafo original.
 - El flujo que estoy mandando de v a u .

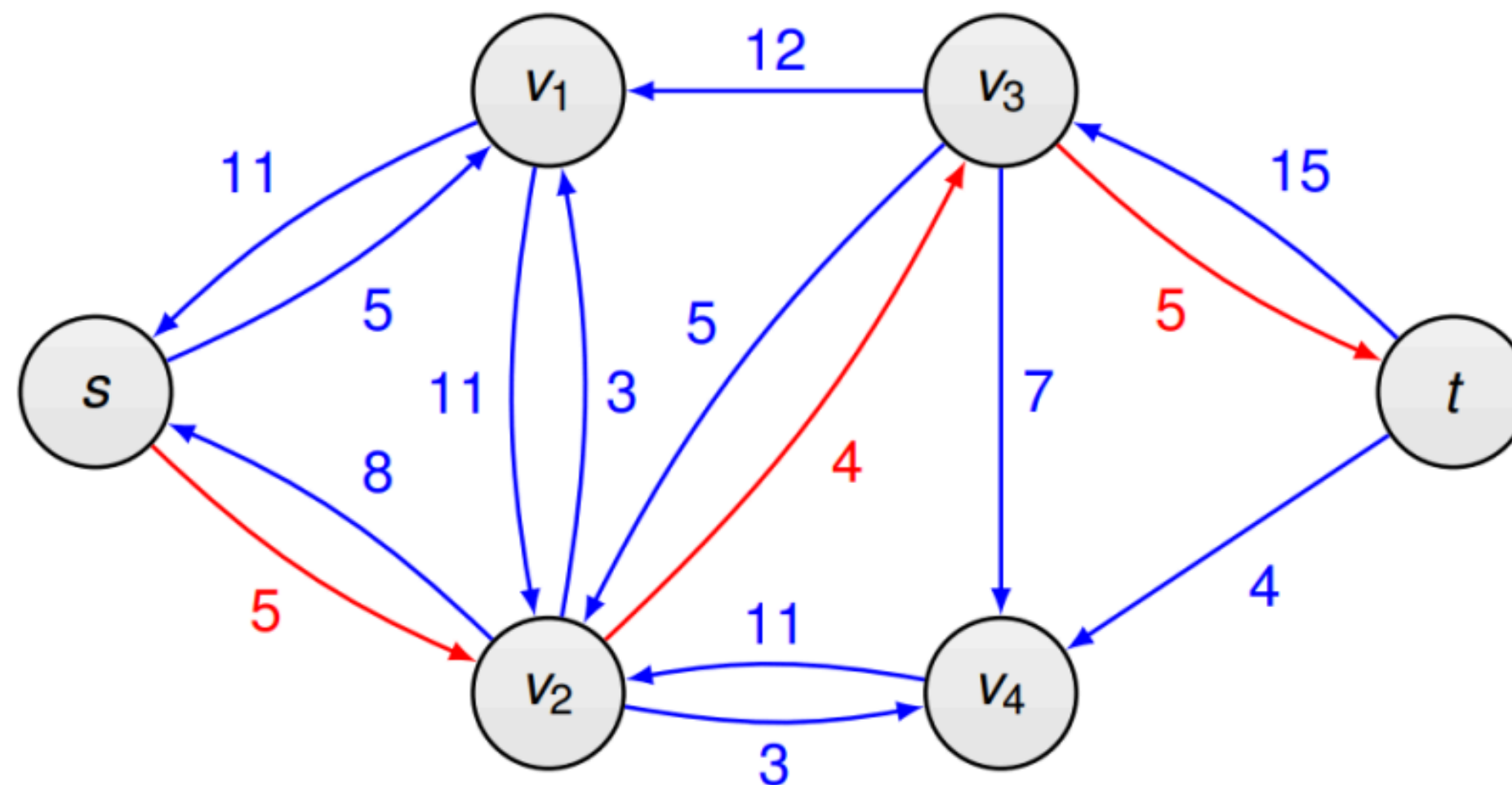
RED RESIDUAL



CAMINO DE AUMENTO

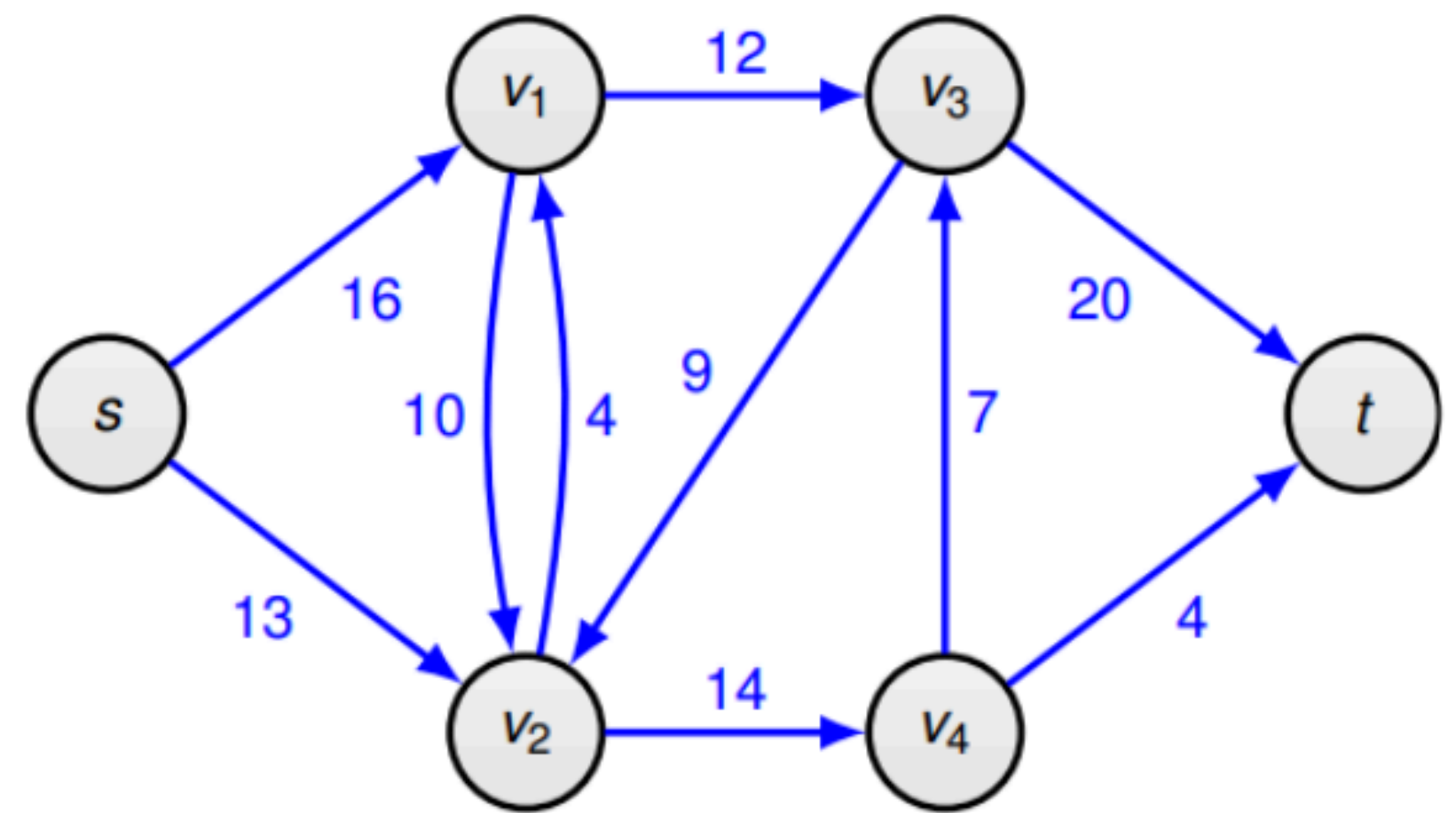
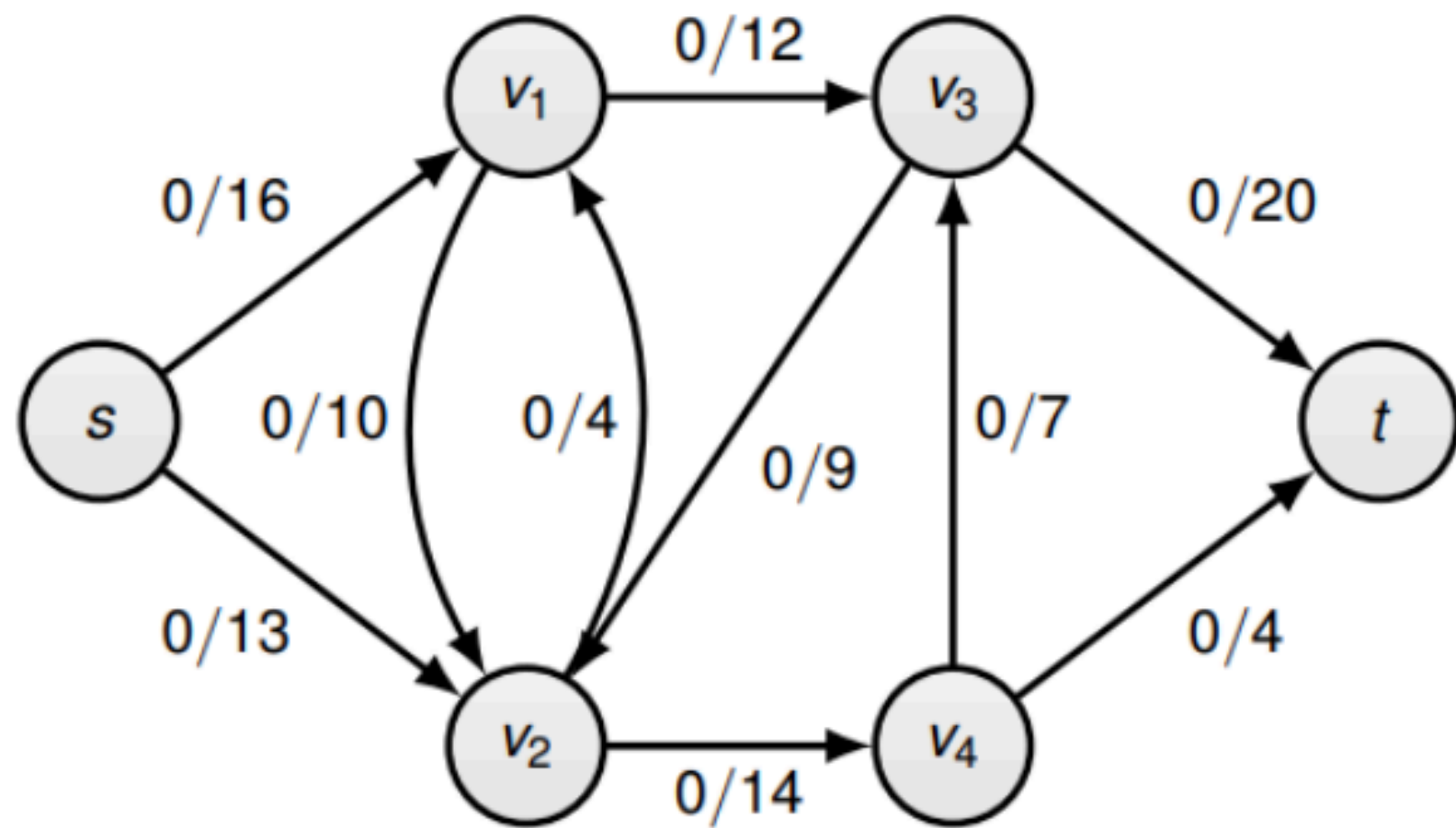
Un camino de aumento es un camino de s a t en la red residual.

Dado un camino de aumento podemos conseguir un flujo mayor mandando todo lo posible por este camino.



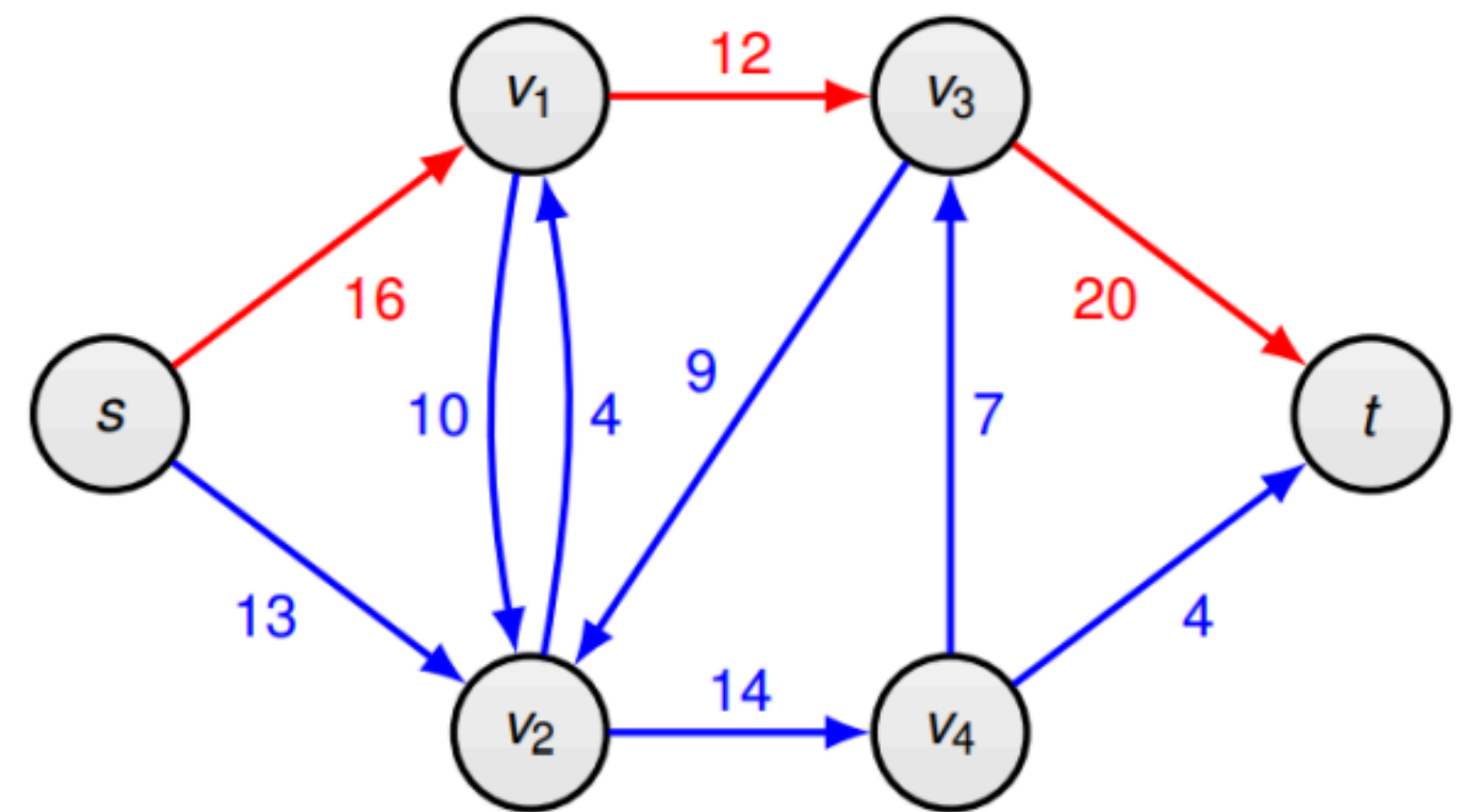
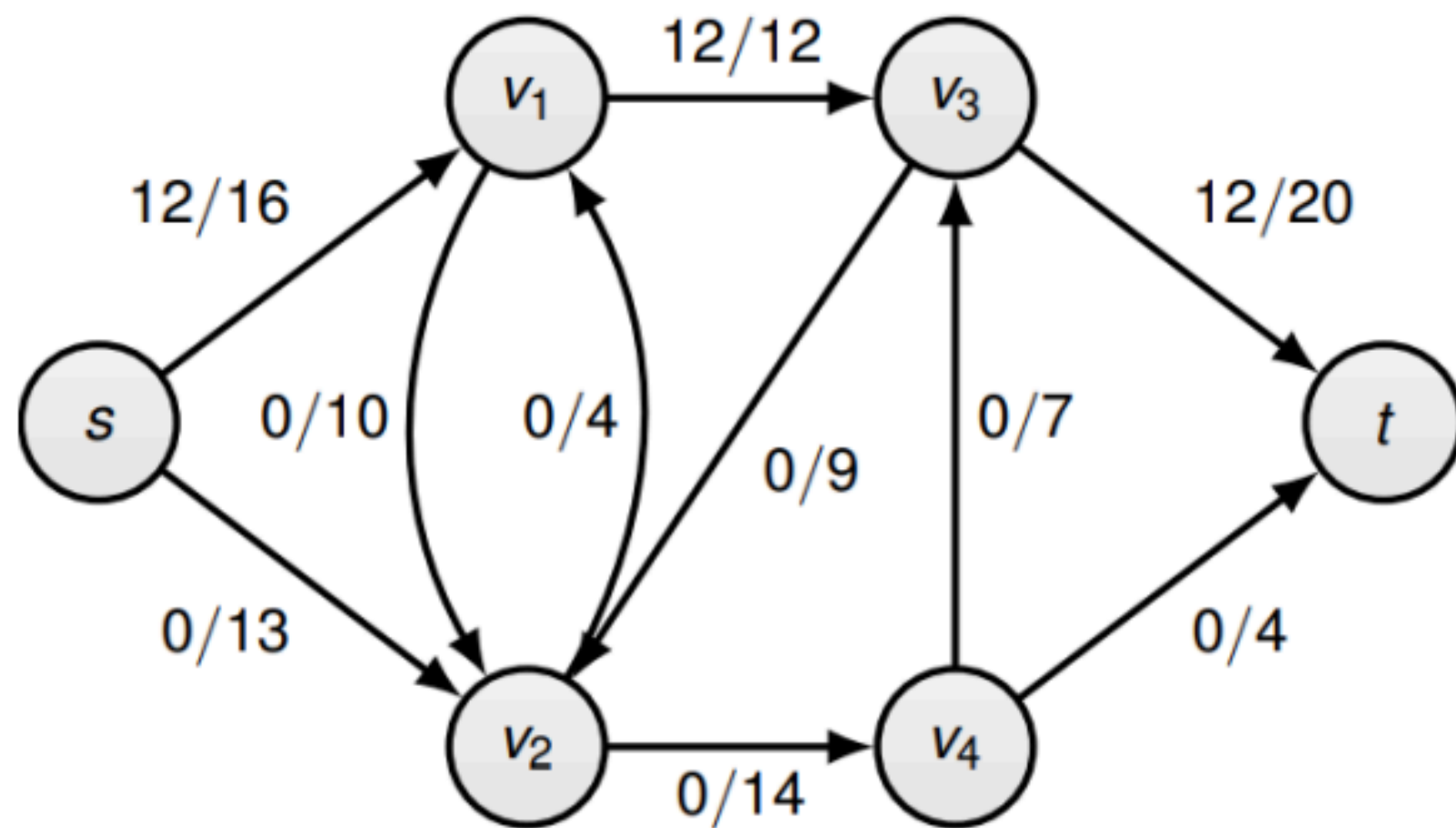
EJEMPLO DE FORD FULKERSON

Empezamos con 0 en todas las aristas (flujo enviado)



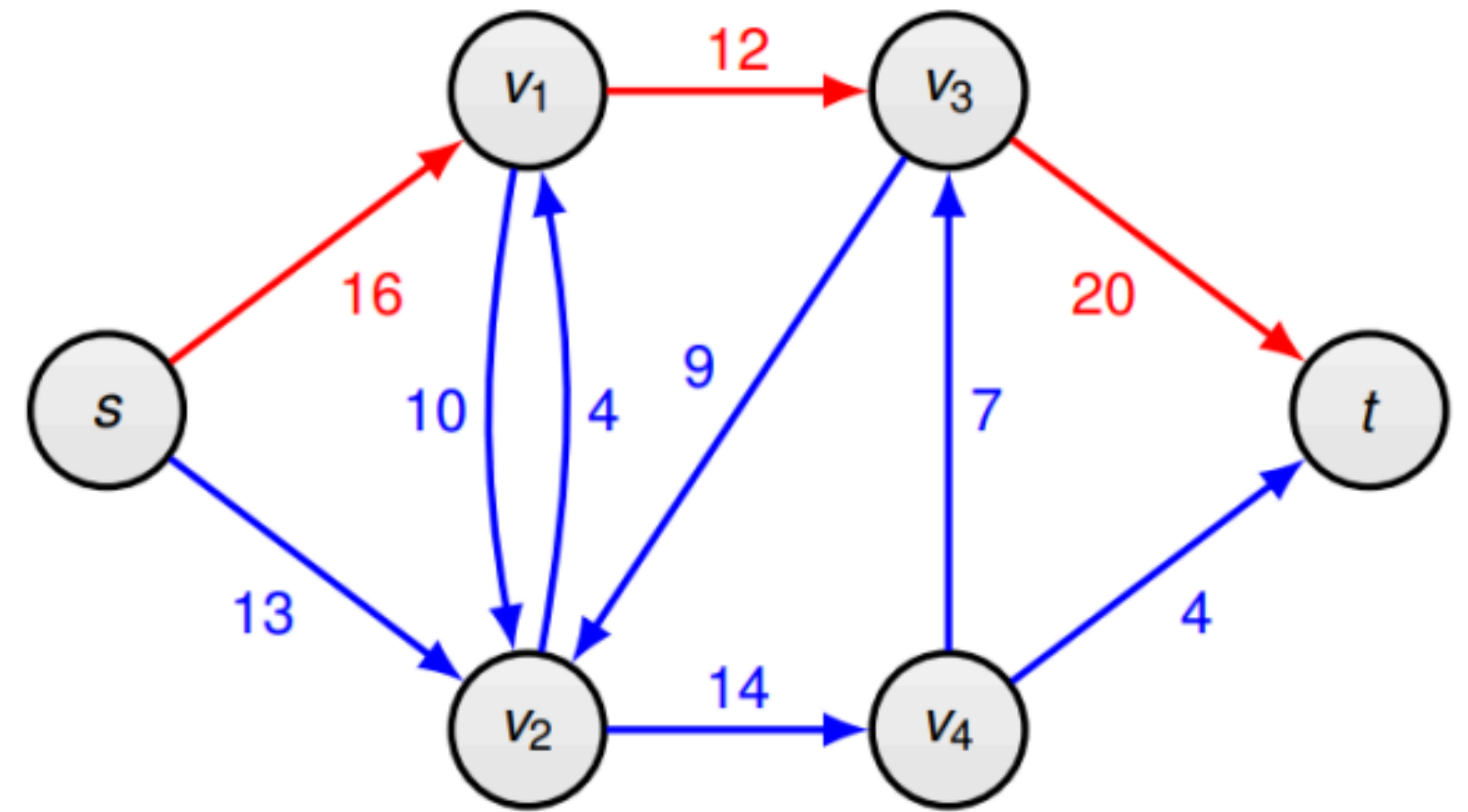
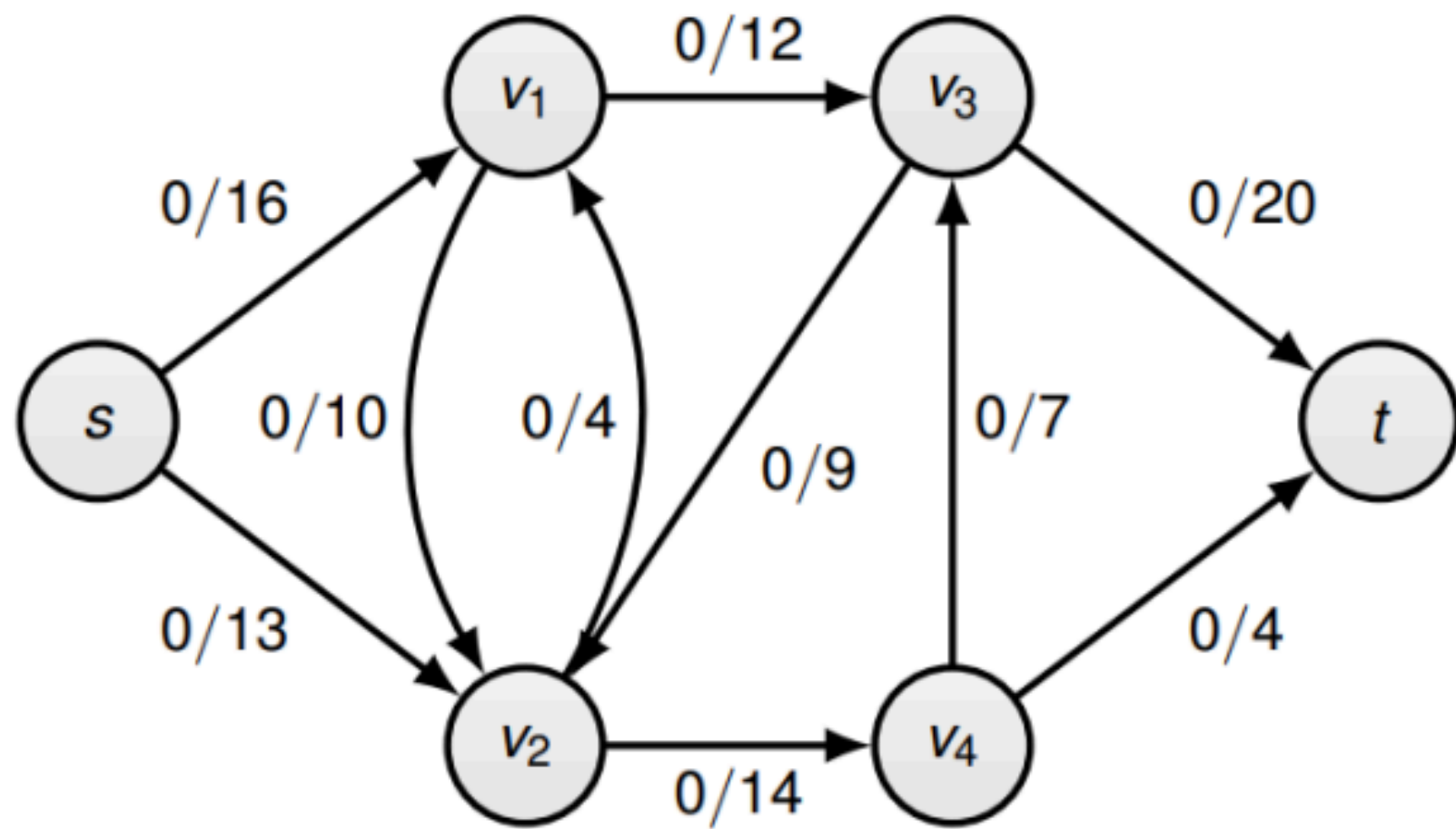
EJEMPLO DE FORD FULKERSON

Mandamos todo lo que podemos por ese camino



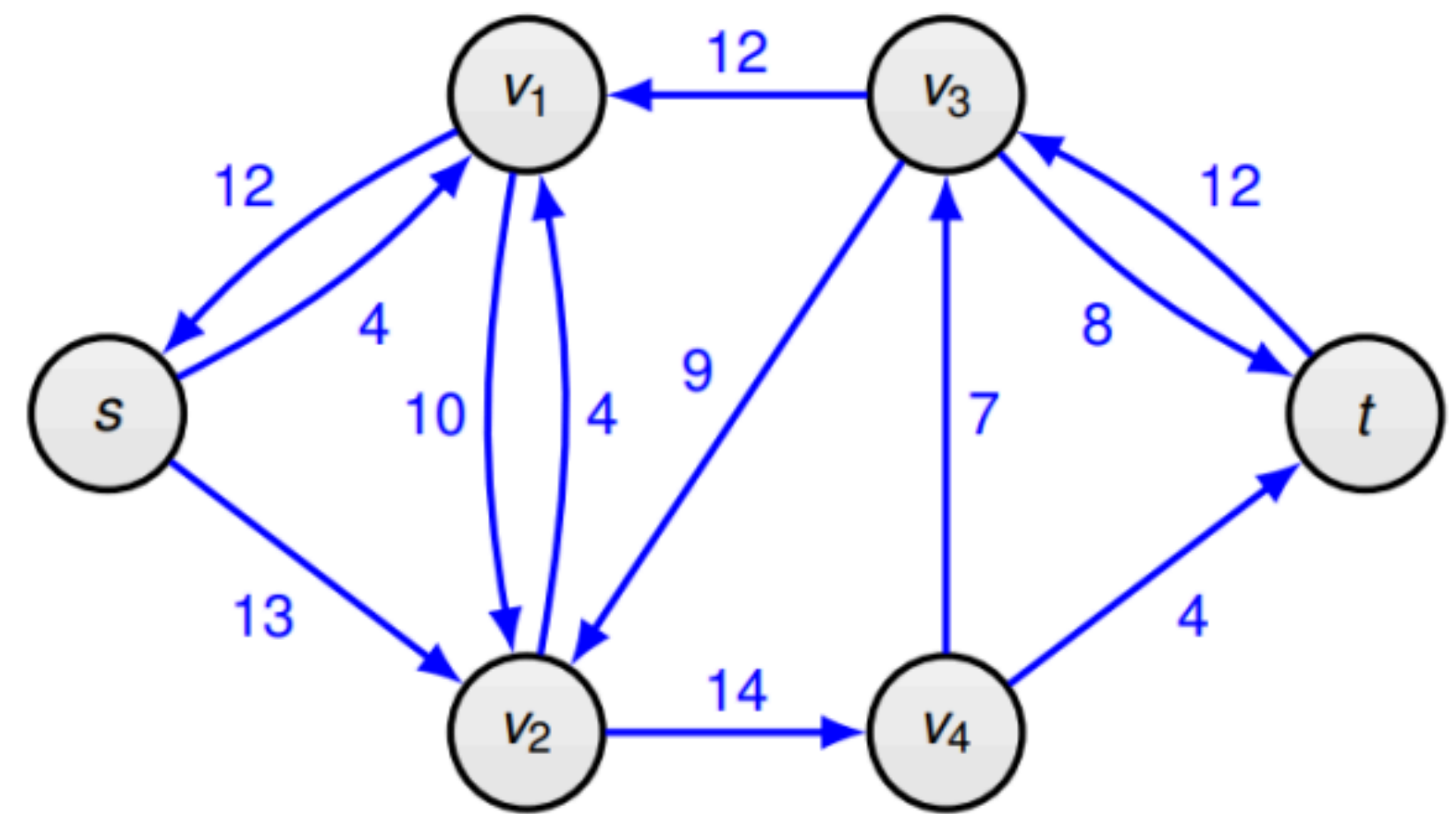
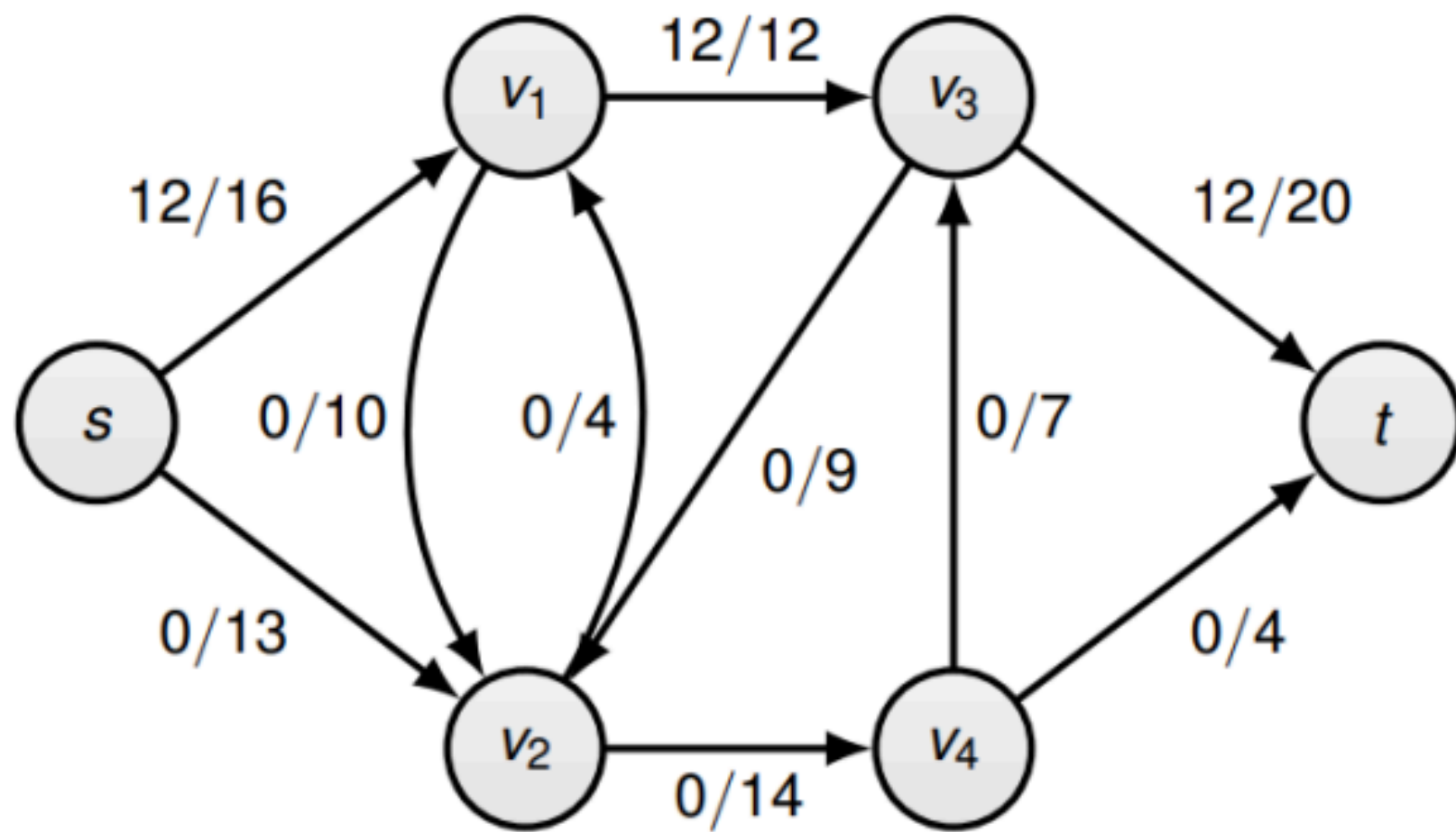
EJEMPLO DE FORD FULKERSON

Encontramos un camino aumentante en la red residual.



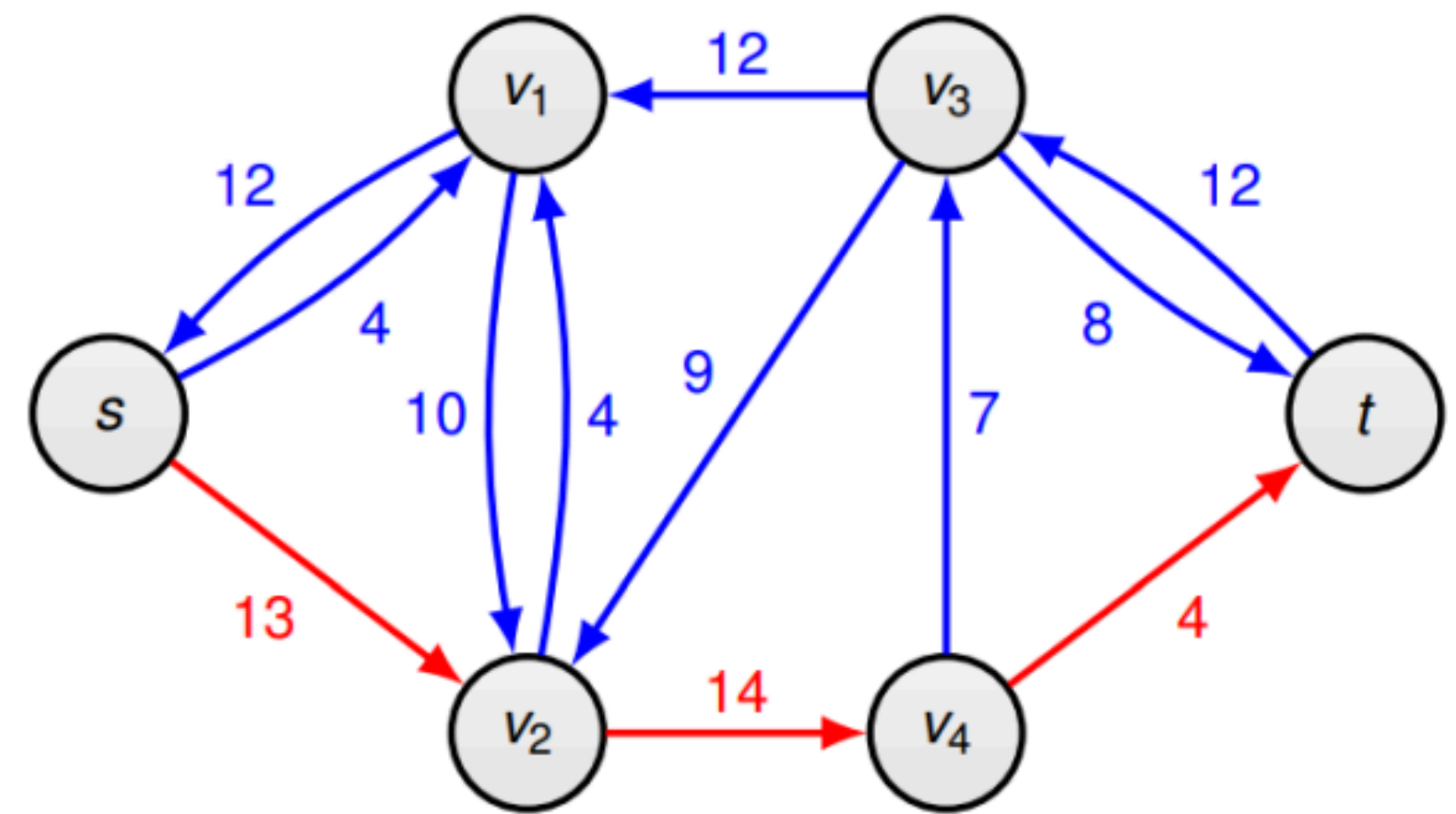
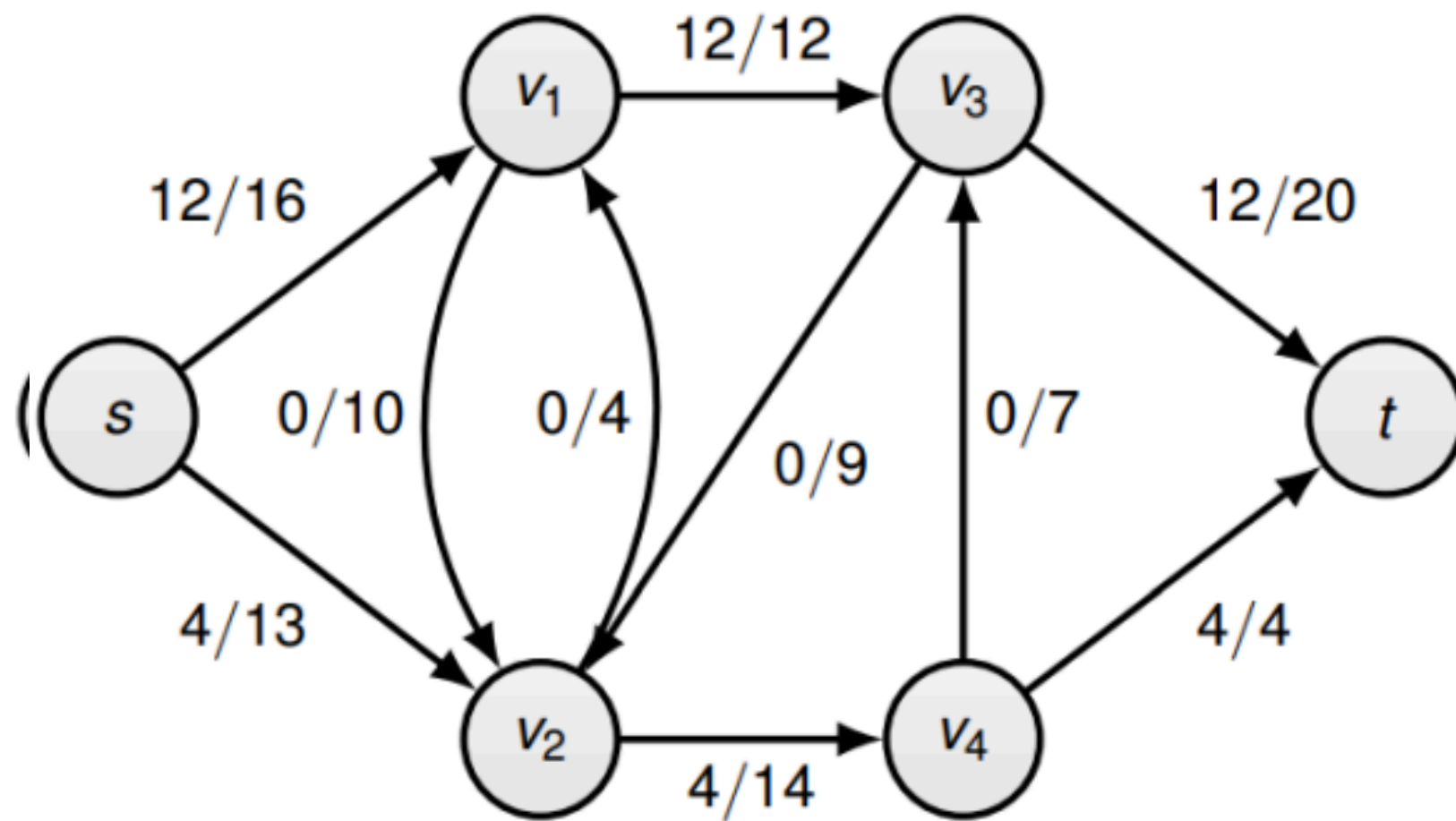
EJEMPLO DE FORD FULKERSON

Actualizamos la red residual.



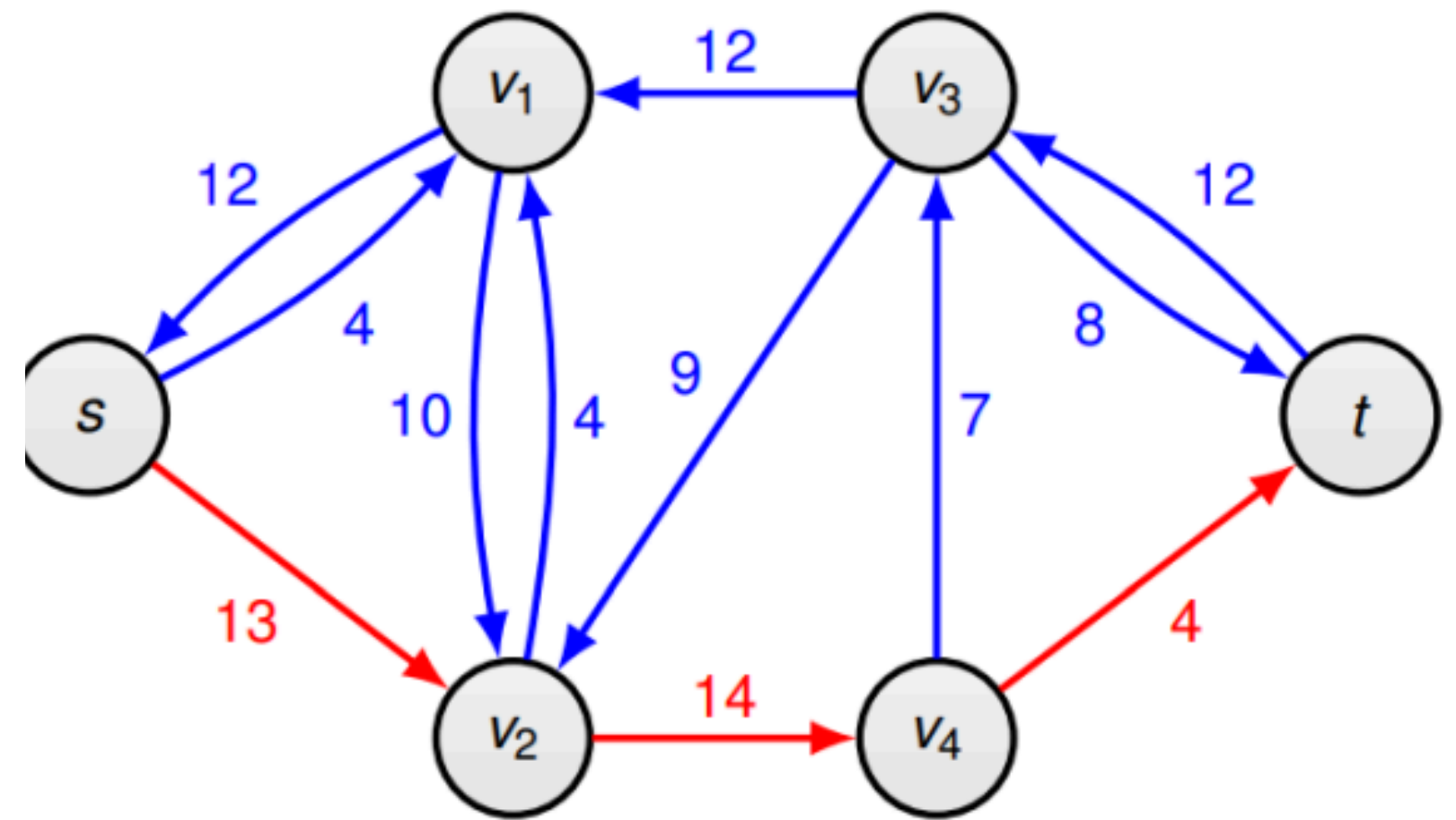
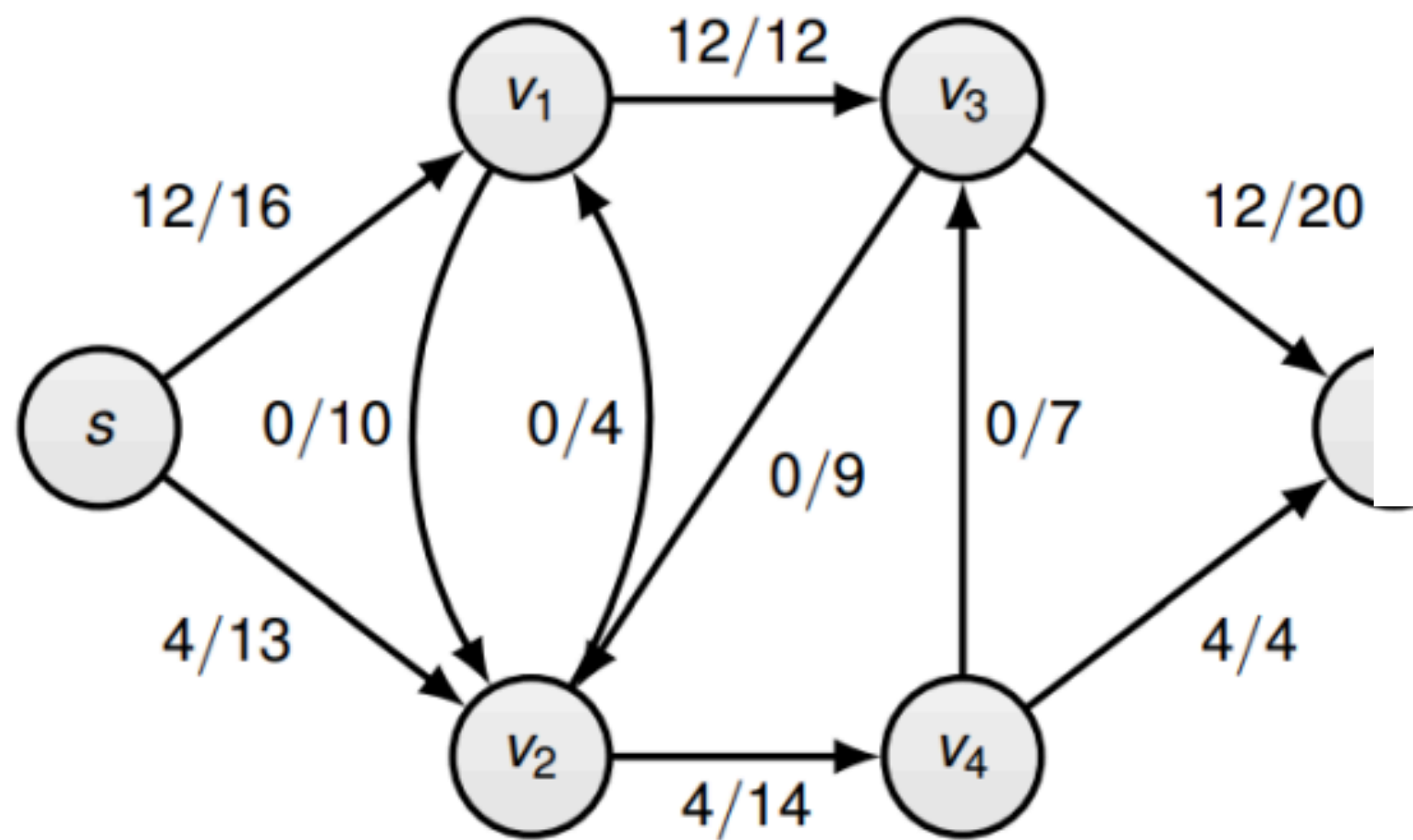
EJEMPLO DE FORD FULKERSON

Volvemos a buscar un camino aumentante en la red residual y mandamos flujo por el.



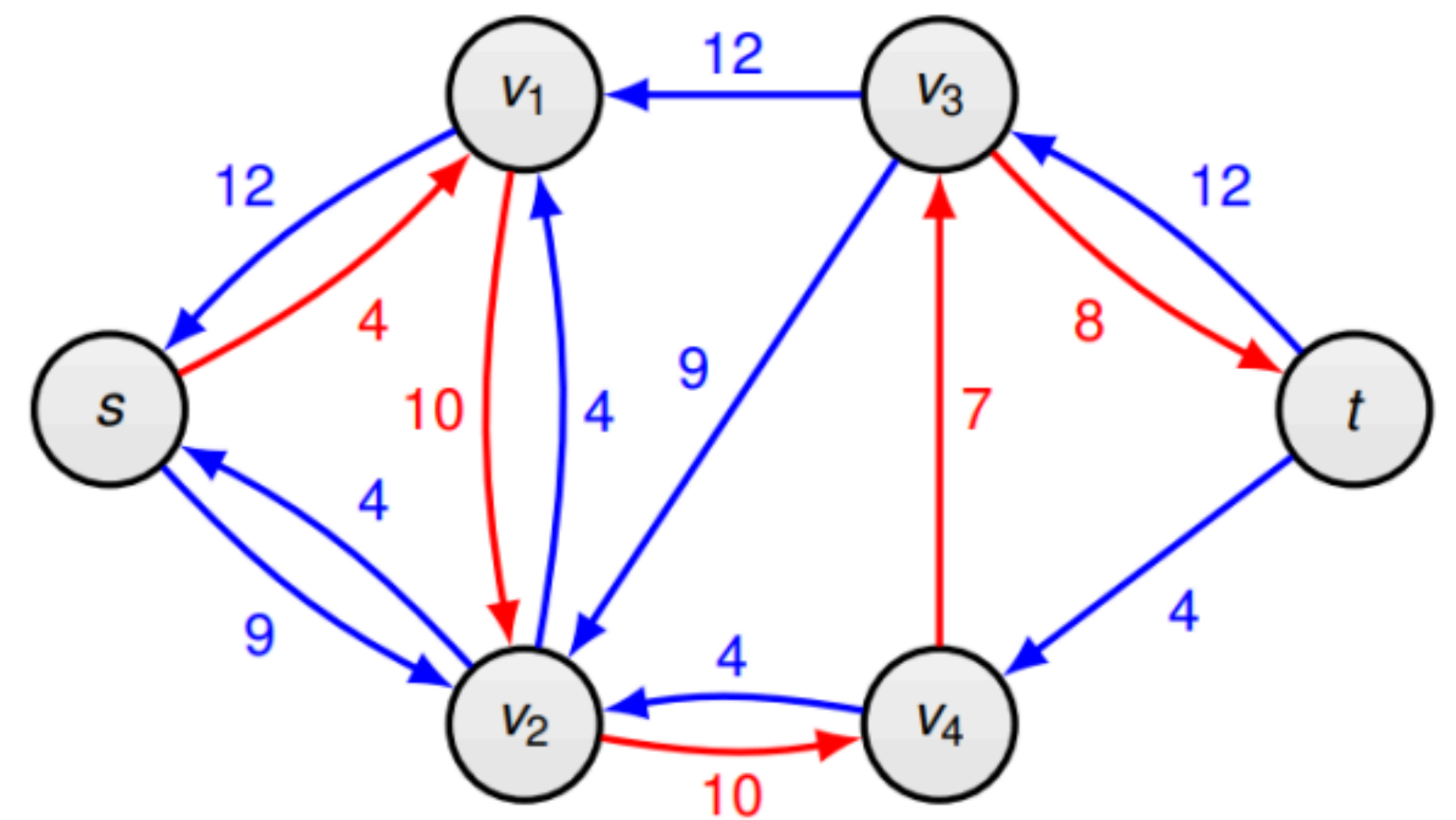
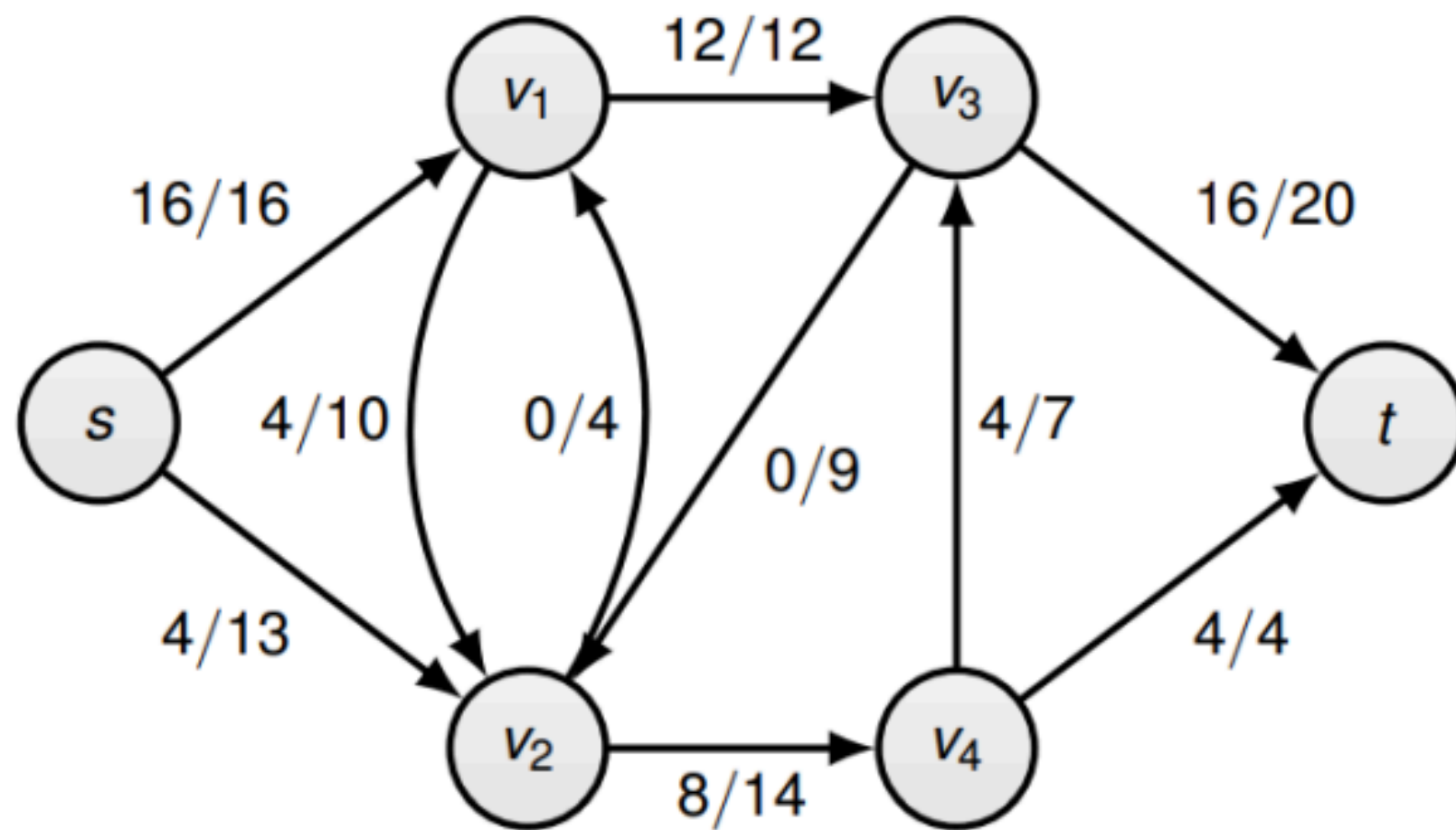
EJEMPLO DE FORD FULKERSON

Volvemos a buscar un camino aumentante en la red residual y mandamos flujo por el.



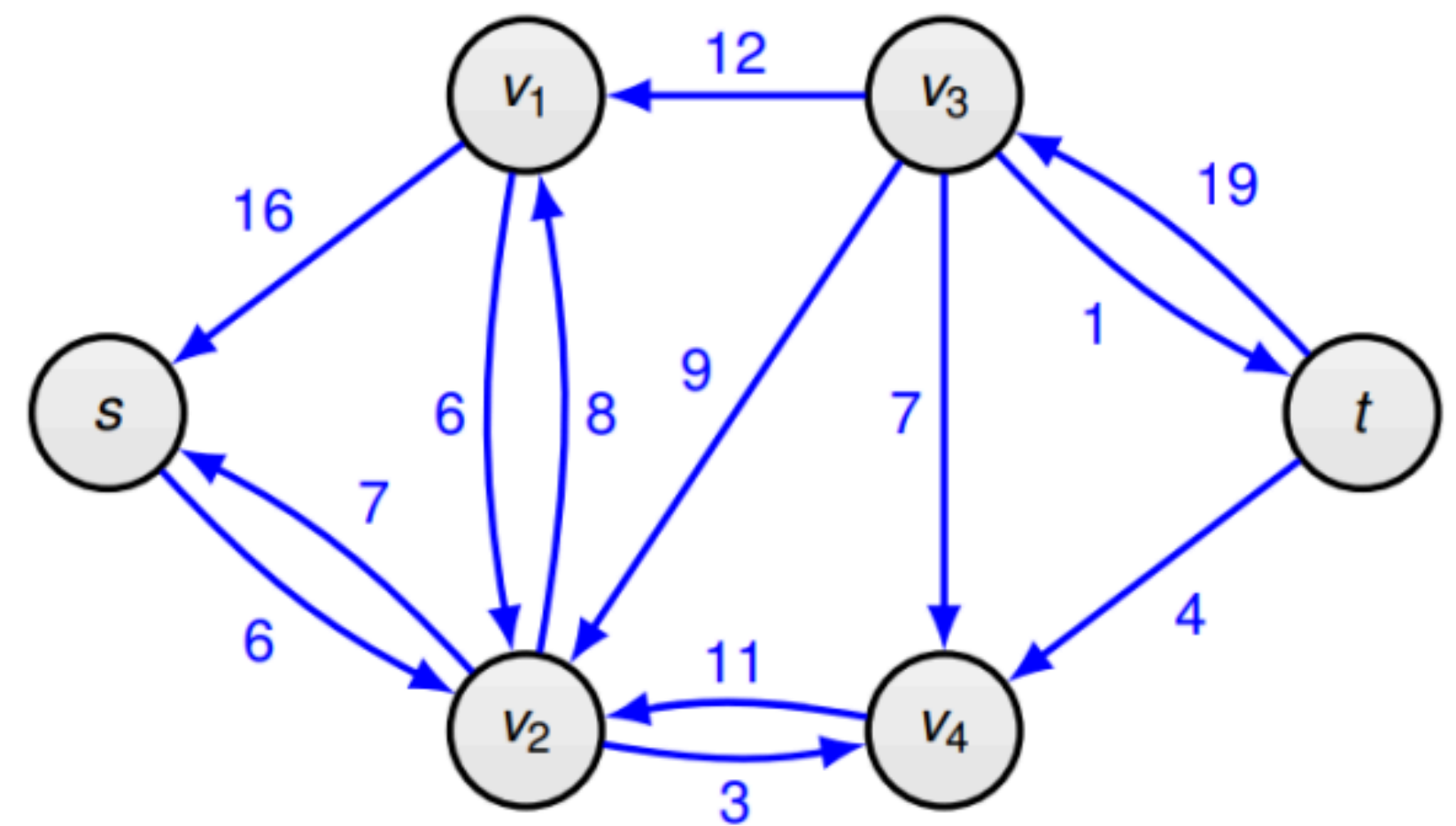
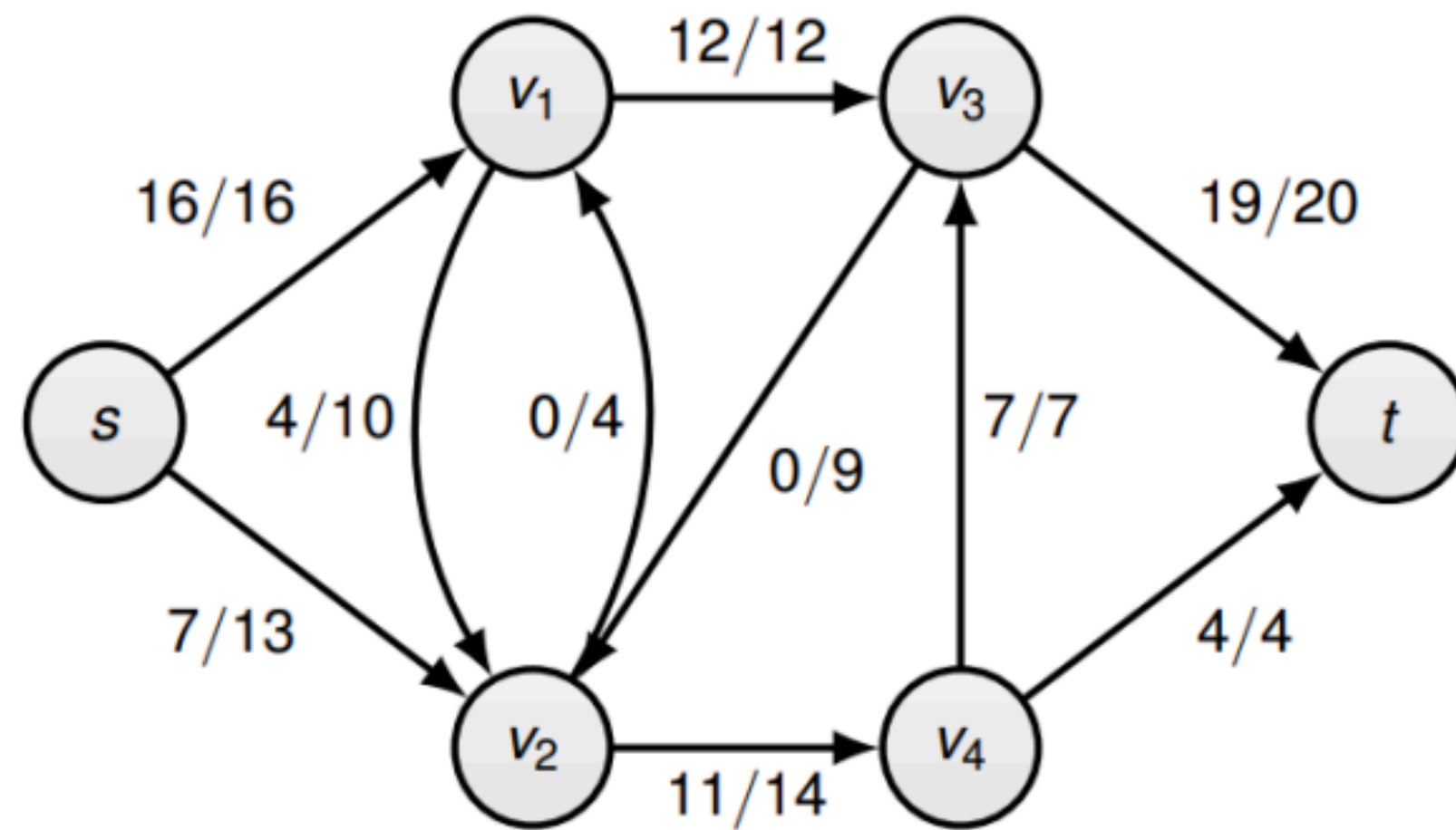
EJEMPLO DE FORD FULKERSON

Actualizamos la red residual y repetimos.



EJEMPLO DE FORD FULKERSON

No hay mas caminos aumentantes en la red residual, paramos.



FLUJO MÁXIMO A MÍNIMO COSTO

Existe una variante muy útil del problema de flujo máximo:

- Para cada arista recibamos un número que represente el **costo de enviar una unidad de flujo** por esa arista (ahora, cada arista tiene 2 números asociados: su capacidad y su costo).
- Definamos el **costo** de un flujo: la suma de $\text{flujo} * \text{costo}$ para cada arista.
- Ahora buscamos, entre todos los flujos máximos posibles, el que tenga menor costo.

ALGORITMO DE DINIC

El algoritmo de Ford Fulkerson es bastante básico. Existe un algoritmo aún más complejo con varias etapas inventado por Dinitz (a veces llamado algoritmo de Dinic). Este algoritmo resuelve flujo máximo de mínimo costo de manera eficiente.

El funcionamiento de este algoritmo es interesante pero fuera del alcance de esta clase. Lo usaremos como una caja negra para resolver el problema de flujo máximo.

COMPLEJIDAD DE DINIC

La complejidad de Dinic es $O(E V^2)$.

Recordar que $O(x)$ significa que el algoritmo corre igual o más rápido que $O(x)$!

Para grafos especiales, existen cotas mejores.

COMPLEJIDAD DE DINIC

Para **redes unitarias**, la cota es $O(E V^{1/2})$

Una **red es unitaria** si:

- Para cada nodo, se cumple alguna de las siguientes dos:
 - Hay 1 sola arista entrante, de capacidad 1.
 - Hay 1 sola arista saliente, de capacidad 1.

COMPLEJIDAD DE DINIC

Para **redes de capacidad unitaria**, la cota es **$O(E V^{2/3})$**

Relajamos “red unitaria” a “red de capacidad unitaria”.

Una **red es de capacidad unitaria** si:

- Para cada nodo, se cumple alguna de las siguientes dos:
 - Todas las aristas entrantes son de capacidad 1.
 - Todas las aristas salientes son de capacidad 1.

PROBLEMA

Ya solo resolver el problema de flujo máximo a mínimo costo es súper útil, pero además podemos resolver otros problemas usando flujo.

- **Tenemos una matriz de $N \times N$, con un 0 ó un 1 en cada posición.**
- **Tenemos $2N$ valores $A_1, A_2, \dots, A_N$, y $B_1, B_2, \dots, B_N$**
- **Queremos que los números en la columna i sumen A_i , y los números de la fila j sumen B_j**
- **¿Cuál es la mínima cantidad de celdas de la matriz que hay que cambiar para obtener lo que queremos?**

PROBLEMA

	1	2	2	3
2	0	0	0	0
2	0	0	1	1
1	0	0	0	0
3	1	1	1	1

PROBLEMA

	1	2	2	3
2	0	1	1	0
2	0	0	1	1
1	0	0	0	1
3	1	1	0	1

SIMPLIFICACIÓN

	1	2	2	3
2	0	0	0	0
2	0	0	0	0
1	0	0	0	0
3	0	0	0	0

**Imaginemos que la matriz contiene solo 0s.
¿Cómo solucionamos el problema?**

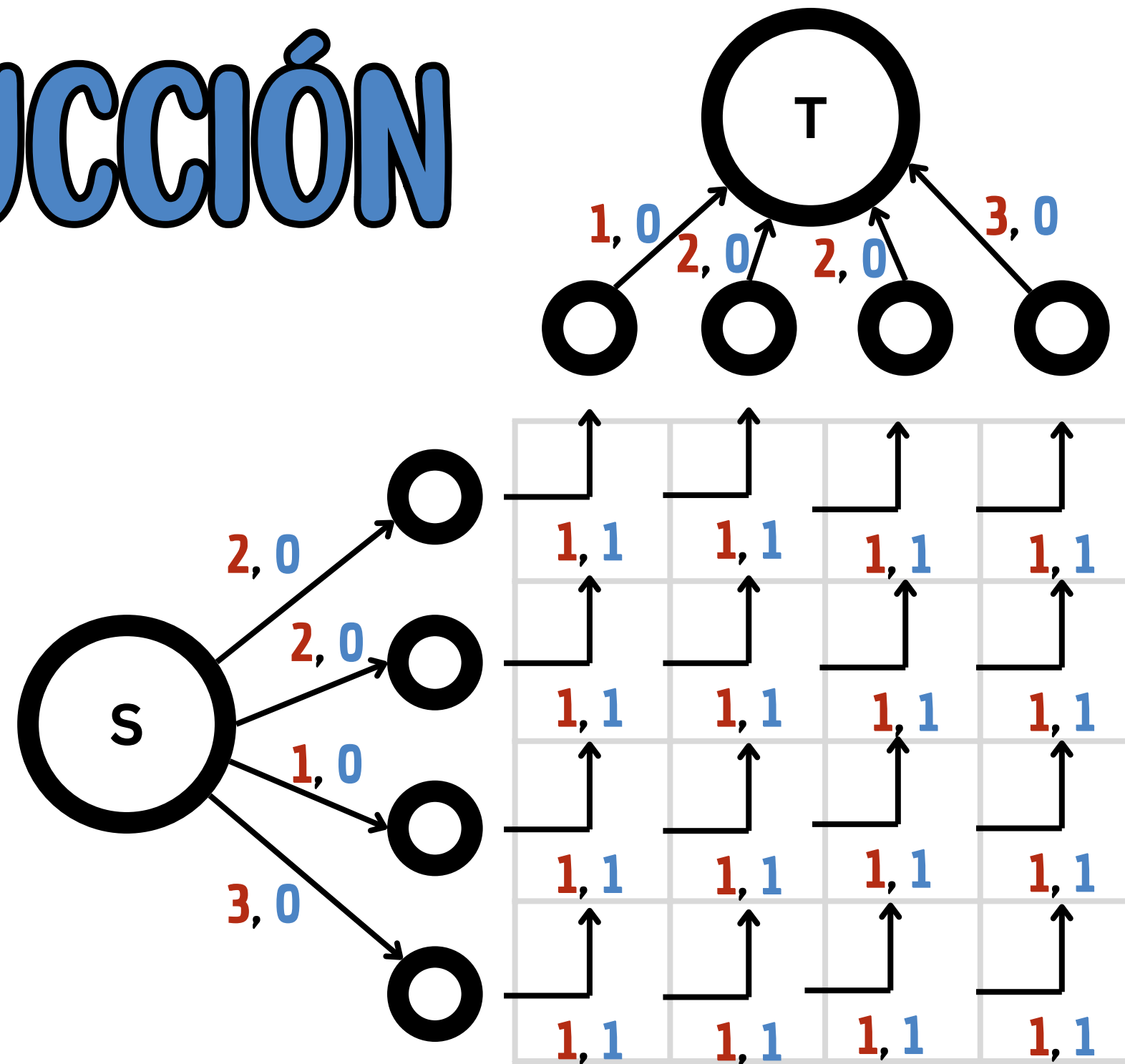
REDUCCIÓN

Resolvamos el problema simplificado construyendo una red de flujo.

La idea es:

- **A partir del problema original, construir una red de flujo.**
- **Encontrar el flujo máximo a mínimo costo en la red.**
- **A partir de la solución, reconstruir la solución al problema original.**

REDUCCIÓN



Pesos de arista:
capacidad, costo

REDUCCIÓN

En la red de flujo que construimos, hay un nodo por cada fila y uno por cada columna.

La idea es que la fila i emana A_i unidades de flujo, y la columna j acepta B_j unidades de flujo.

Hay N^2 aristas, una entre cada par (i, j) .

En nuestra construcción, que fluya flujo entre i y j significa que hay un 1 en la posición (i, j) .

- El costo del flujo será la cantidad de 1s que pongamos.
- Si el flujo máximo es $A_1 + A_2 + \dots + A_N$, entonces el problema es resoluble, y se cumple la condición de filas y columnas.

COMPLEJIZACIÓN

Ahora, veamos como solucionar el problema cuando comenzamos con una matriz que puede contener 1s.

El problema es similar, solo que ahora en algunas posiciones poner 1s es gratis, y colocar 0s en esas posiciones tiene costo.

Podemos cambiar nuestra interpretación:

Interpretación vieja: Colocar un 1 cuesta \$0. Colocar un 0 cuesta \$1.

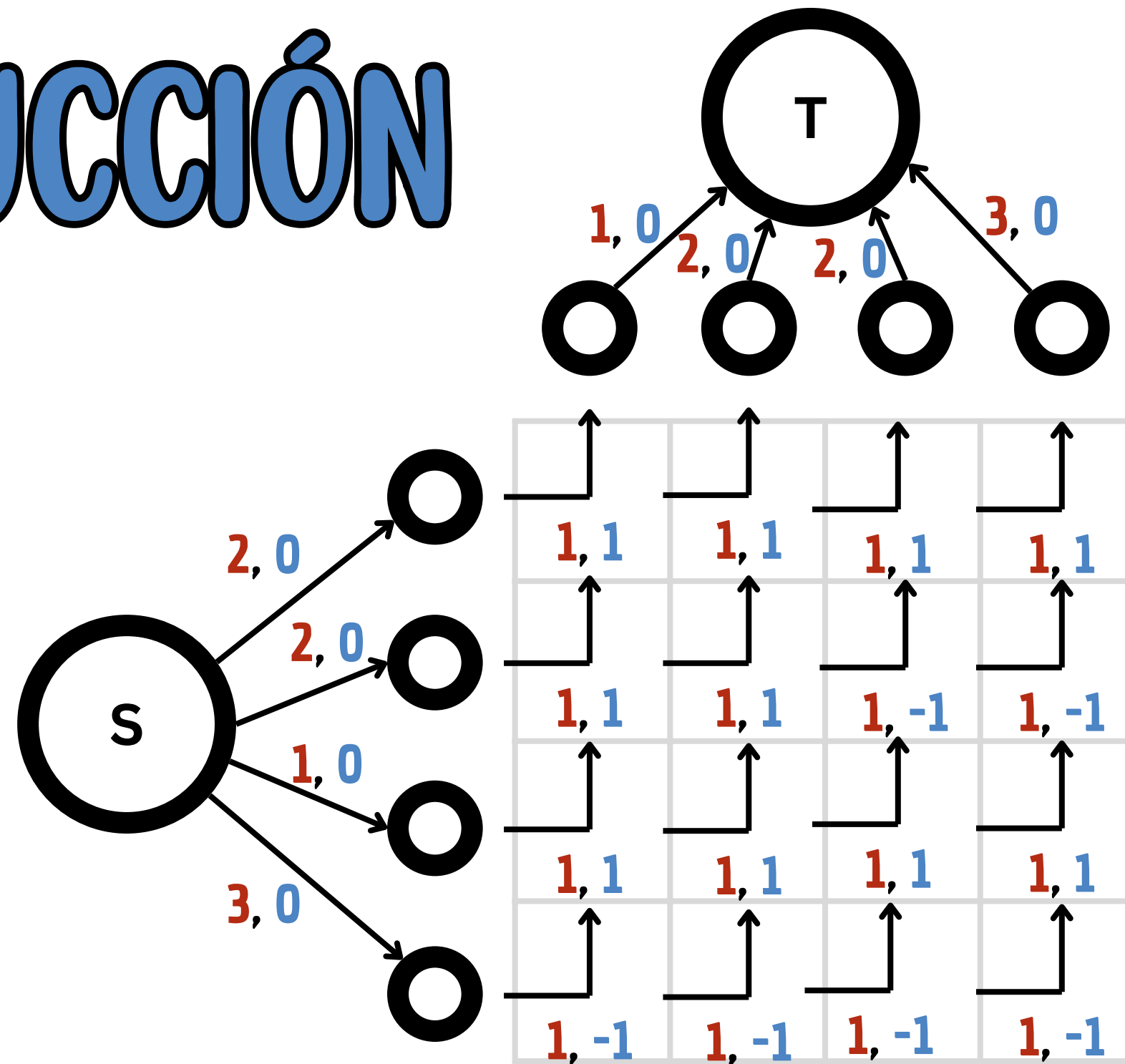
Interpretación nueva: Colocar un 1 cuesta \$-1. Colocar un 0 cuesta \$0.

COMPLEJIZACIÓN

En resumen, el algoritmo sería:

- Reemplazar todos los valores de la matriz por 0s.
- Armar la red de flujo que hacíamos antes, pero las posiciones que originalmente tenían un 1 escrito, hacemos que tengan costo $\$-1$.
- Buscamos el flujo máximo a costo mínimo.
- Las aristas por las que pasa flujo representan un 1.
- Tenemos que sumar un costo base para balancear que colocar 1s ahora cuesta $\$-1$.

REDUCCIÓN



Costo base: 6

Pesos de arista:
capacidad, costo

OPTIMIZACIÓN LINEAL

Cuando definimos el problema de flujo formalmente, usamos ecuaciones del tipo:

$$\begin{array}{ll} 0 \leq \text{flujo} \leq \text{capacidad} & \text{para toda arista} \\ \text{flujo entrante} - \text{flujo saliente} = 0 & \text{para todo nodo} \end{array}$$

Un problema de flujo no es más que muchas de estas ecuaciones, con las variables siendo la cantidad de flujo en cada arista.

¡Es un sistema de ecuaciones lineal!

Luego, queremos optimizar la suma de los flujos bajo estas restricciones.

PROBLEMA DUAL

Un teorema que queda fuera del alcance de este curso, es que cualquier problema de optimización lineal (como lo es flujo) tiene un “problema dual”.

El problema dual se resuelve exactamente igual al problema original, pero la interpretación es distinta.

El problema dual de flujo máximo se llama “mínimo corte”.

MÍNIMO CORTE

El problema de mínimo corte consiste en:

- **Dado:** Un grafo dirigido con pesos no-negativos en las aristas.
- **Buscar:** Un subconjunto de las aristas.
- **Minimizar:** La suma de los pesos de las aristas elegidas.

MÍNIMO CORTE

El problema de mínimo corte consiste en:

- **Dado:** Un grafo dirigido con pesos no-negativos en las aristas.
- **Buscar:** Un subconjunto de las aristas.
- **Minimizar:** La suma de los pesos de las aristas elegidas.

Para resolver el problema de mínimo corte, basta con reinterpretar los pesos de las aristas como capacidades, correr el algoritmo de máximo flujo, y luego cortar las aristas saturadas (ie. el subconjunto de aristas a cortar son las que su flujo sea igual a su capacidad).

MÍNIMO CORTE

El problema de mínimo corte consiste en:

- **Dado:** Un grafo dirigido con pesos no-negativos en las aristas.
- **Buscar:** Un subconjunto de las aristas.
- **Minimizar:** La suma de los pesos de las aristas elegidas.

**Para resolver el problema de mínimo corte, basta con reinterpretar los pesos de las aristas como capacidades, correr el algoritmo de máximo flujo, y luego cortar las aristas saturadas (ie. el subconjunto de aristas a cortar son las que su flujo sea igual a su capacidad).
(Es bastante mágico)**

BIBLIOGRAFIA DE CODIGOS

Dinic (Max Flow) : <https://github.com/Marceantasy/apuntes-MEMoir-del-pudu/blob/main/graphs/Dinic.cpp>

MinCost - MaxFlow: <https://github.com/Marceantasy/apuntes-MEMoir-del-pudu/blob/main/graphs/MinCostMaxFlow.cpp>